ACHARYA
BENGALURU, INDIA
WHERE THE WORLD COMES TO LEARN
www.acharya.ac.in

Operating Systems
21CS44

Mrs. Sneha N P

Assistant Professor

Department of CSE

Textbooks:

ACHARYA
BENGALURU, INDIA

Click to Edit

Prescribed Text Books :

Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, Operating System Principles 7th edition, Wiley-India, 2006.

Reference Books:

1. Ann McHoes Ida M Flynn, Understanding Operating System, Cengage Learning, 6th Edition.

2. D.M Dhamdhere, Operating Systems: A Concept Based Approach 3rd Ed, McGraw Hill, 2013.

3. P.C.P. Bhatt, An Introduction to Operating Systems: Concepts and Practice 4th Edition,PHI(EEE),

2014.

4. William Stallings Operating Systems: Internals and Design Principles, 6th Edition, Pearson.

Course Objectives:

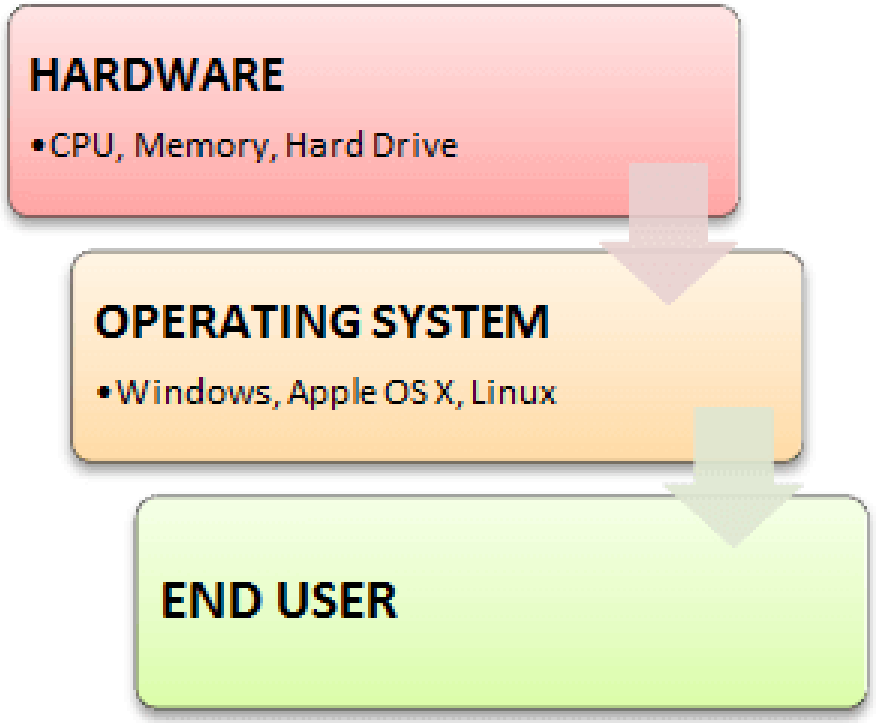
On Completion of the course, the students will be able to	
21CS44.1	Demonstrate the need for OS and different types of OS
21CS44.2	Apply suitable techniques for management of different resources
21CS44.3	Use processor, memory, storage and file system commands
21CS44.4	Realize the different concepts of OS in platform of usage through case studies

Course Outcomes:

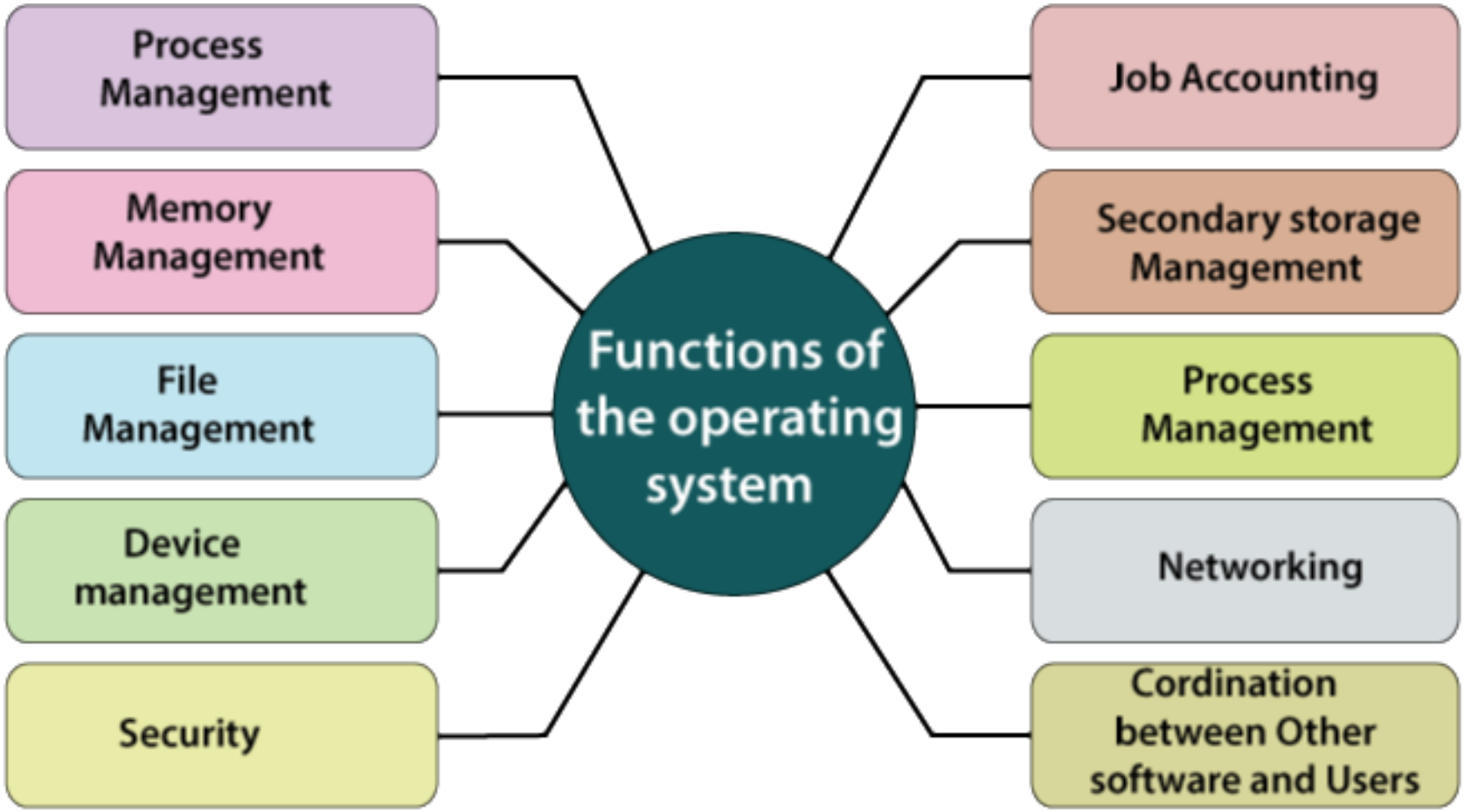
On Completion of the course, the students will be able to	
21CS44.1	Illustrate the fundamentals of the operating system, its various types, its importance in computer systems and to understand different management, security and multi-threaded systems.
21CS44.2	Analyze and compare different scheduling algorithms, paging concepts, memory and disk allocation methods.
21CS44.3	Demonstrate operating system's resource management techniques, deadlock management techniques, memory management techniques.
21CS44.4	Apply theory and implementation of processes, resource control (concurrency etc.), physical and virtual memory, scheduling, I/O and files.

What is an operating system?

- An Operating System (OS) is an interface between a computer user and computer hardware



Operating System - Functions



Operating System - Functions

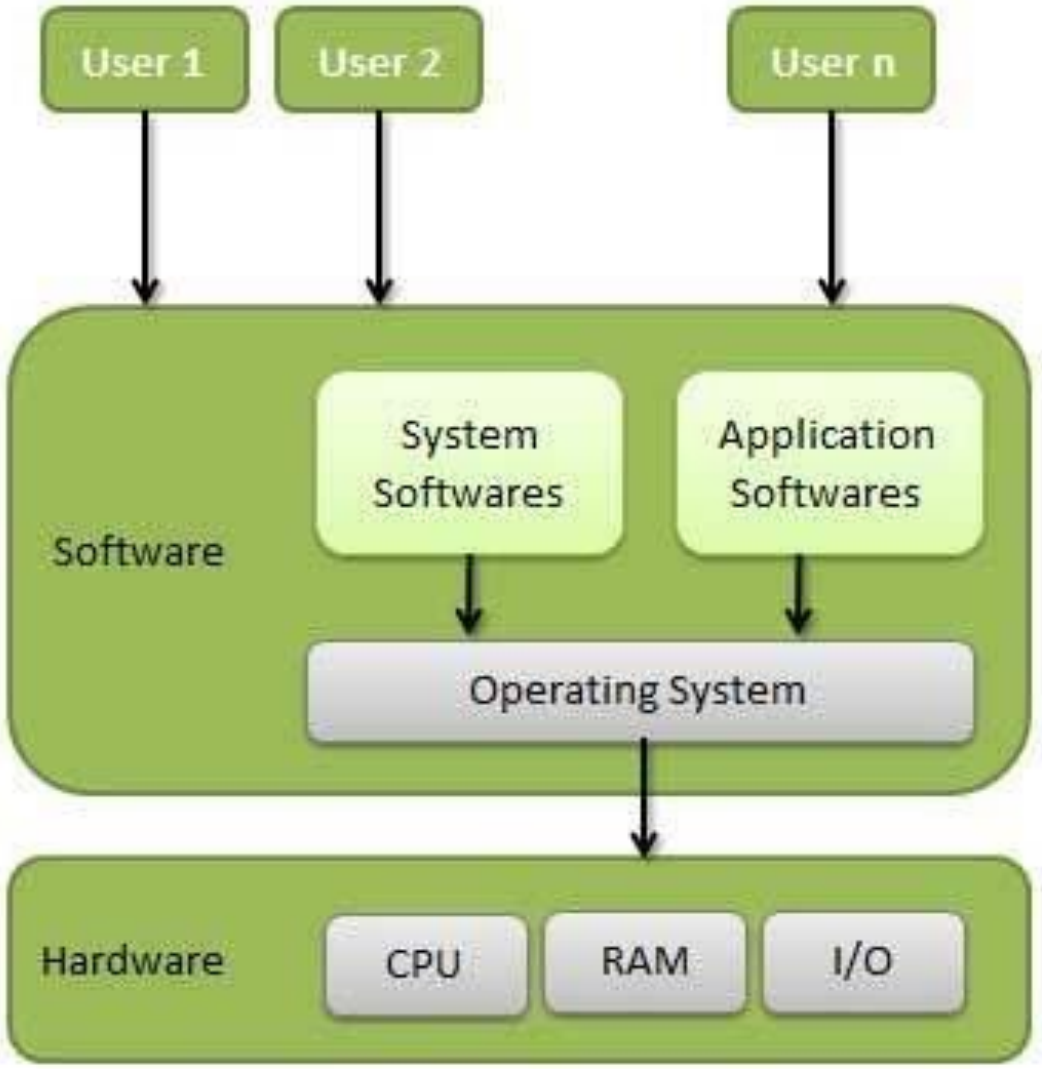
ACHARYA
BENGALURU, INDIA

Click to Edit

An operating system is a software which performs all the basic tasks like

- file management,
- memory management,
- process management,
- handling input and output, and
- controlling peripheral devices such as disk drives and printers.

Computer System components:



Computer System components:

- **Computer Users** are the users who use the overall computer system.
- **Application Softwares** are the softwares which users use directly to perform different activities. These softwares are simple and easy to use like Browsers, Word, Excel, different Editors, Games etc. These are usually written in high-level languages, such as Python, Java and C++.
- **System Softwares** are the softwares which are more complex in nature and they are more near to computer hardware. These software are usually written in low-level languages like assembly language and includes Operating Systems (Microsoft Windows, macOS, and Linux), Compiler, and Assembler etc.
- **Computer Hardware** includes Monitor, Keyboard, CPU, Disks, Memory, etc.

ACHARYA
BENGALURU, INDIA

Click to Edit

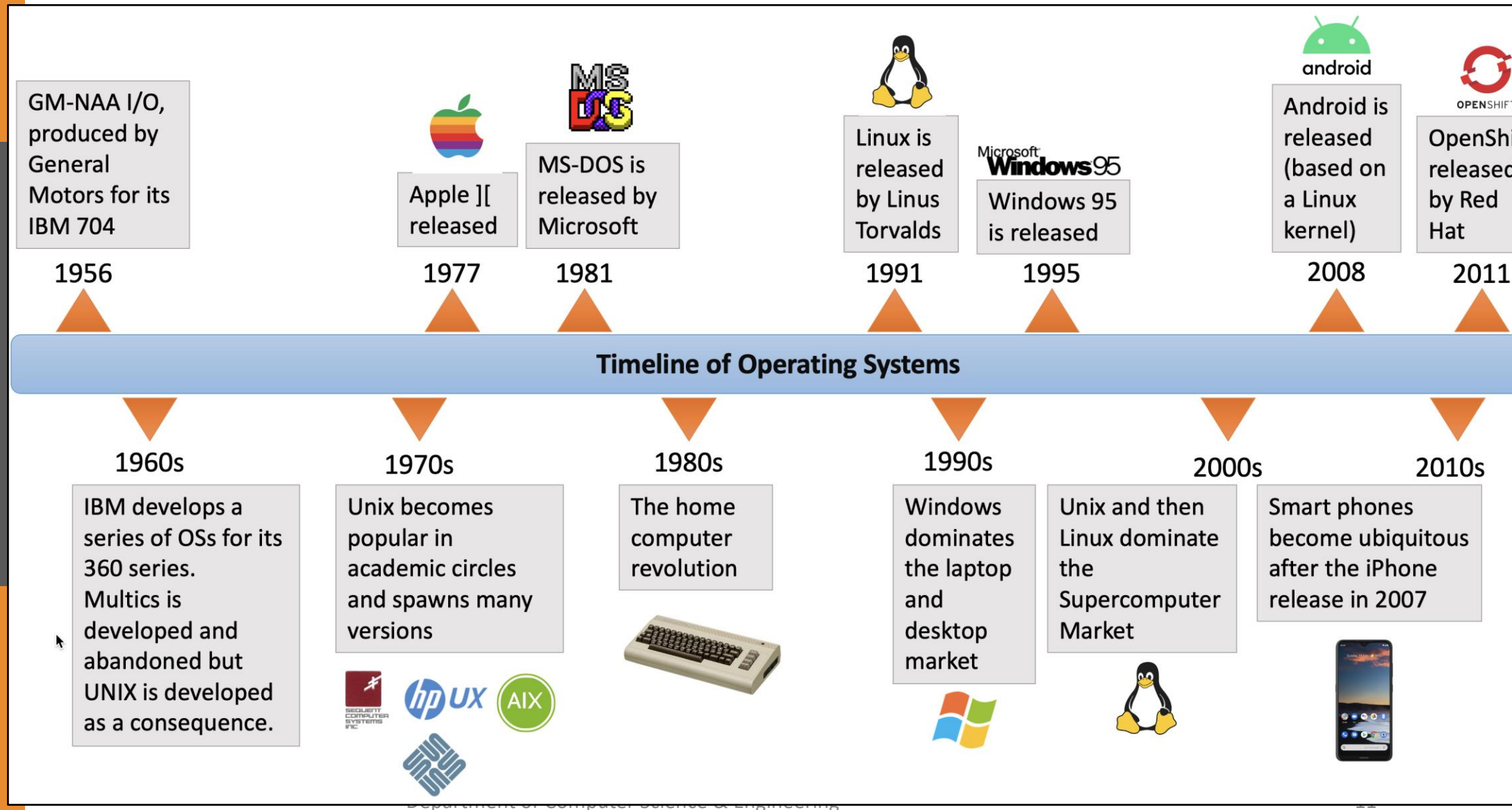
Operating System - Examples



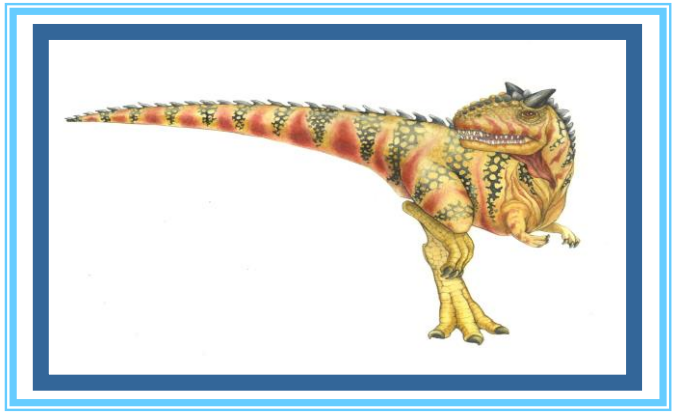
ACHARYA
BENGALURU, INDIA

Click to Edit

Operating Systems - History



Chapter 1: Introduction



Silberschatz, Galvin and Gagne ©2009

Chapter 1: Introduction



Click to Edit

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations
- Process Management
- Memory Management
- Storage Management
- Protection and Security
- Distributed Systems
- Special-Purpose Systems
- Computing Environments
- Open-Source Operating Systems

Objectives

- To provide a grand tour of the major operating systems components
- To provide coverage of basic computer system organization

ACHARYA
BENGALURU, INDIA

Click to Edit

What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware

- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

ACHARYA
BENGALURU, INDIA

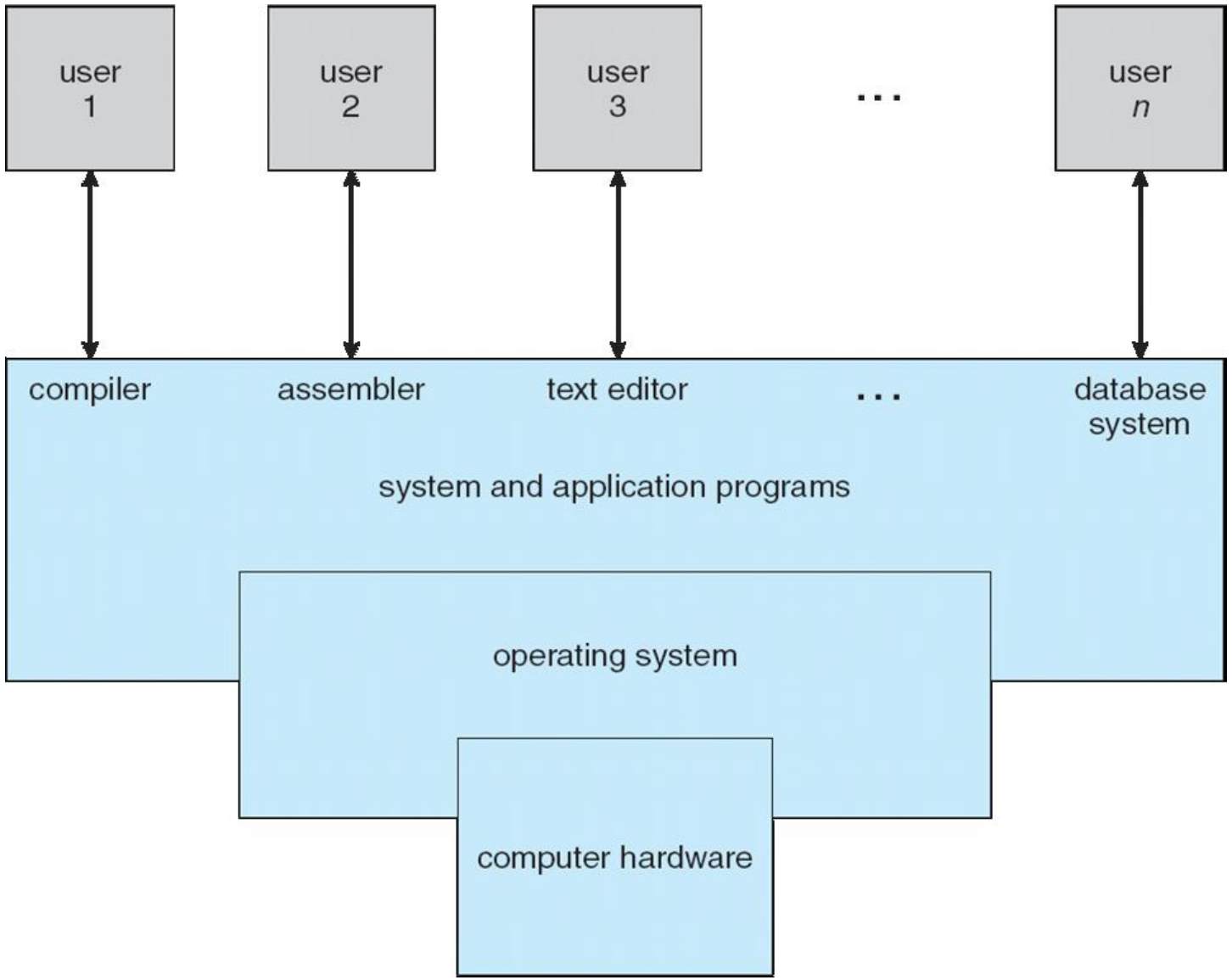
Click to Edit

Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

ACHARYA
BENGALURU, INDIA
Click to Edit

Four Components of a Computer System



ACHARYA
BENGALURU, INDIA

Click to Edit

What Operating Systems Do



Click to Edit

- Depends on the point of view
- Users want convenience, **ease of use**
  - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
- Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- **Handheld computers** are resource poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles

Operating System Definition

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer

ACHARYA
BENGALURU, INDIA

Click to Edit

Operating System Definition (Cont.)

- No universally accepted definition
- "Everything a vendor ships when you order an operating system" is good approximation
  - But varies wildly
- "The one program running at all times on the computer" is the **kernel**. Everything else is either a system program (ships with the operating system) or an application program.

ACHARYA
BENGALURU, INDIA
Click to Edit

Computer Startup

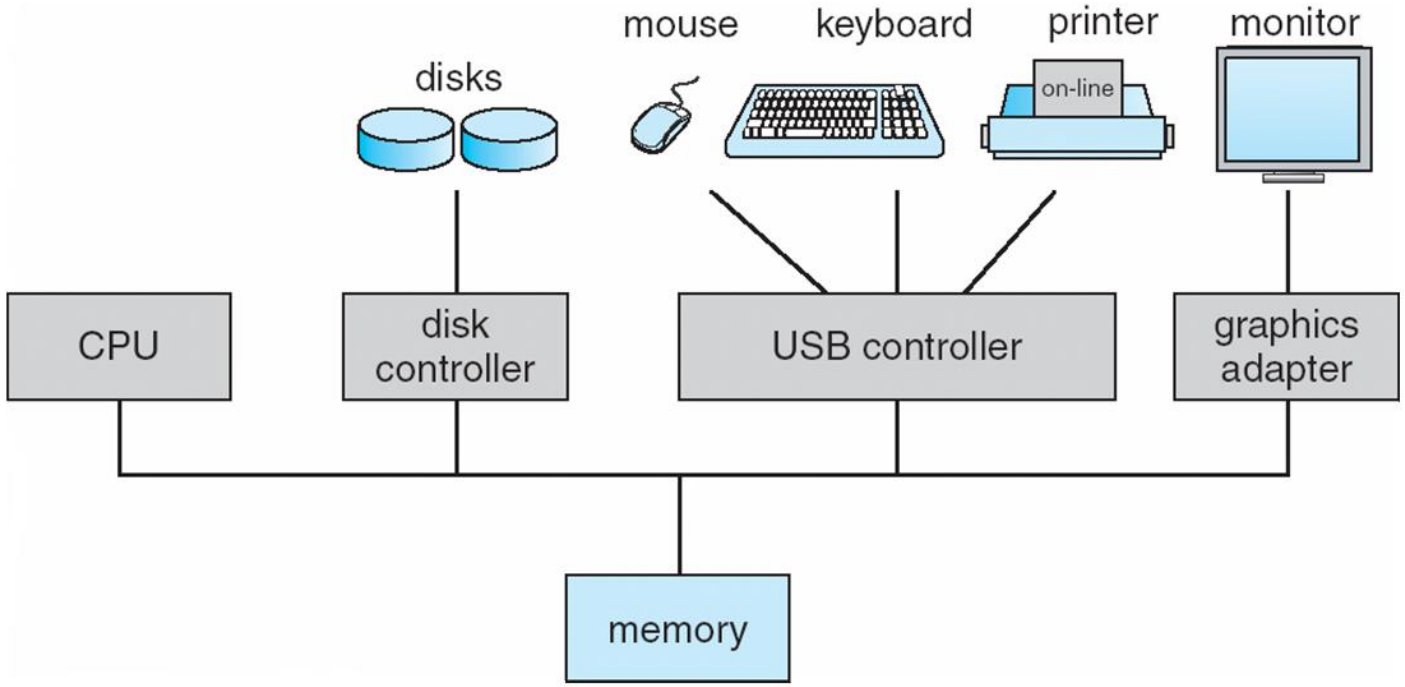
- **bootstrap program** is loaded at power-up or reboot
  - Typically stored in ROM or EPROM, generally known as **firmware**
  - Initializes all aspects of system
  - Loads operating system kernel and starts execution

ACHARYA
BENGALURU, INDIA

Click to Edit

Computer System Organization

- **Computer-system operation**
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



ACHARYA
BENGALURU, INDIA

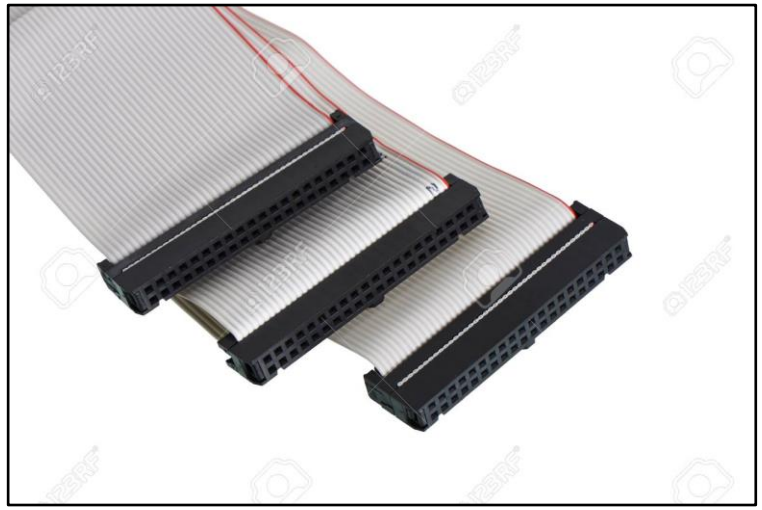
Click to Edit

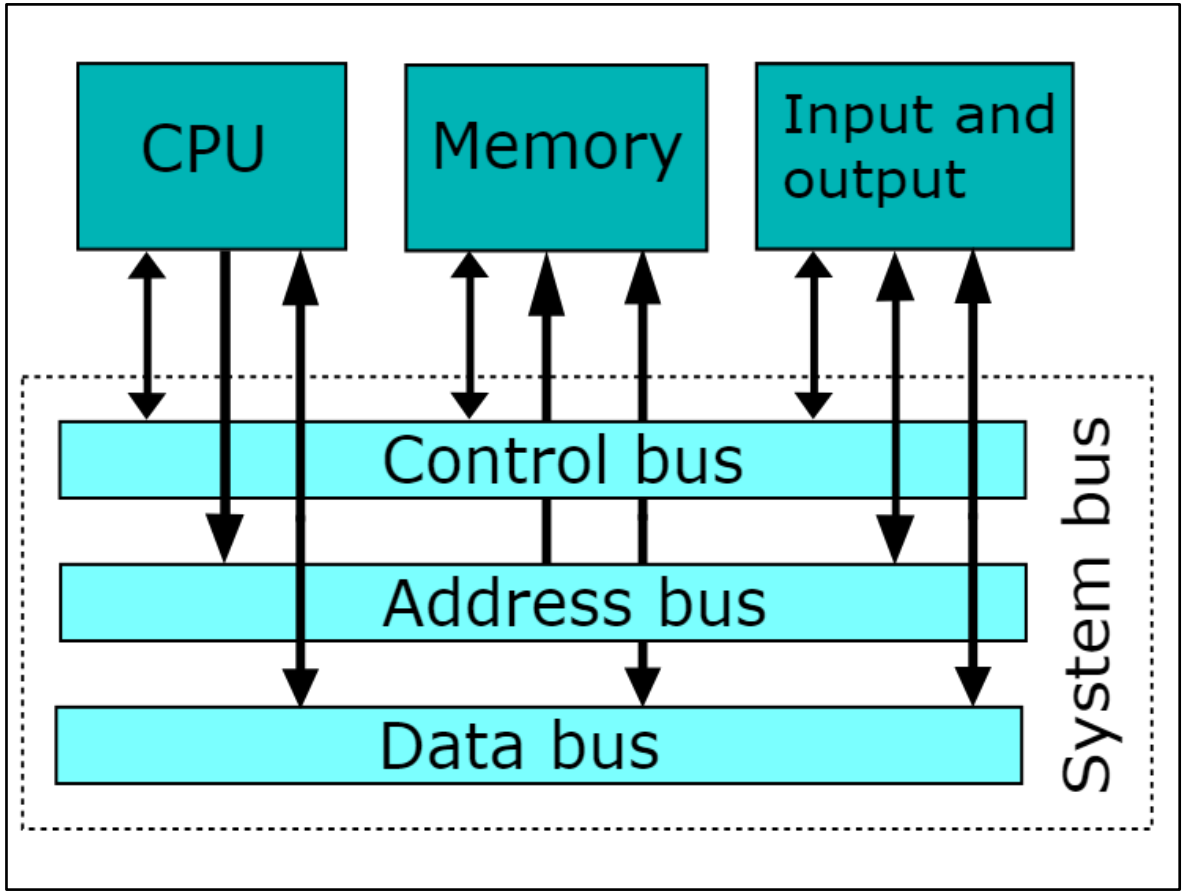
Computer System Organization

- **Computer-system operation**

ACHARYA
BENGALURU, INDIA

Click to Edit





Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

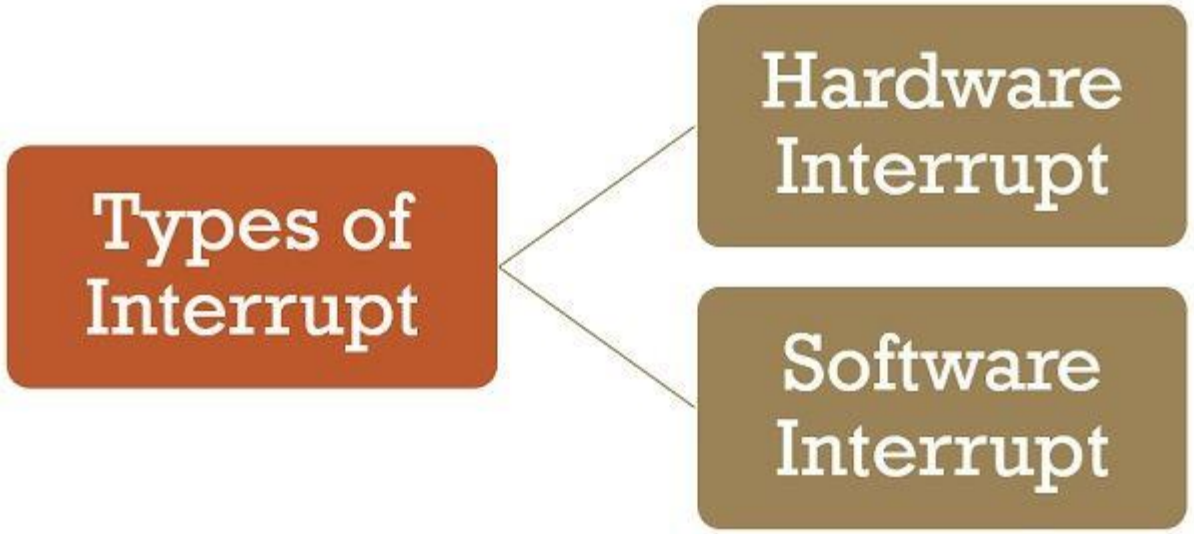
ACHARYA
BENGALURU, INDIA
Click to Edit

Computer-System Operation

ACHARYA
BENGALURU, INDIA

Click to Edit

- Occurrence of an event is signaled by an **interrupt.**
- Separate segments of code determine what action should be taken for each type of interrupt- **Interrupt service routine**

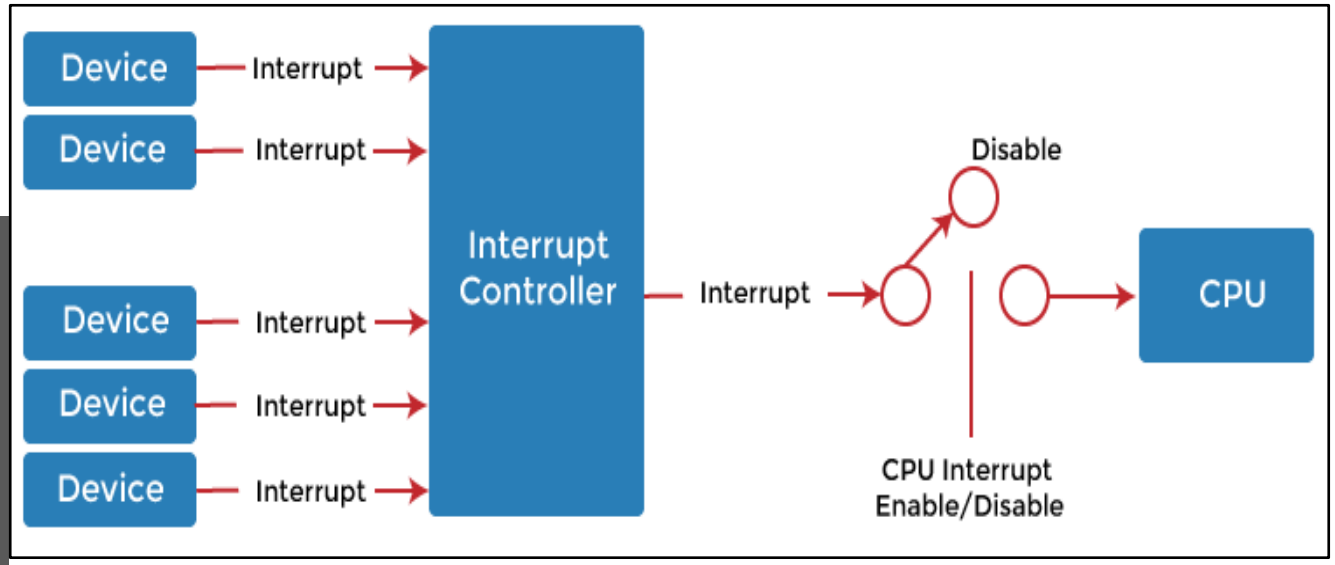


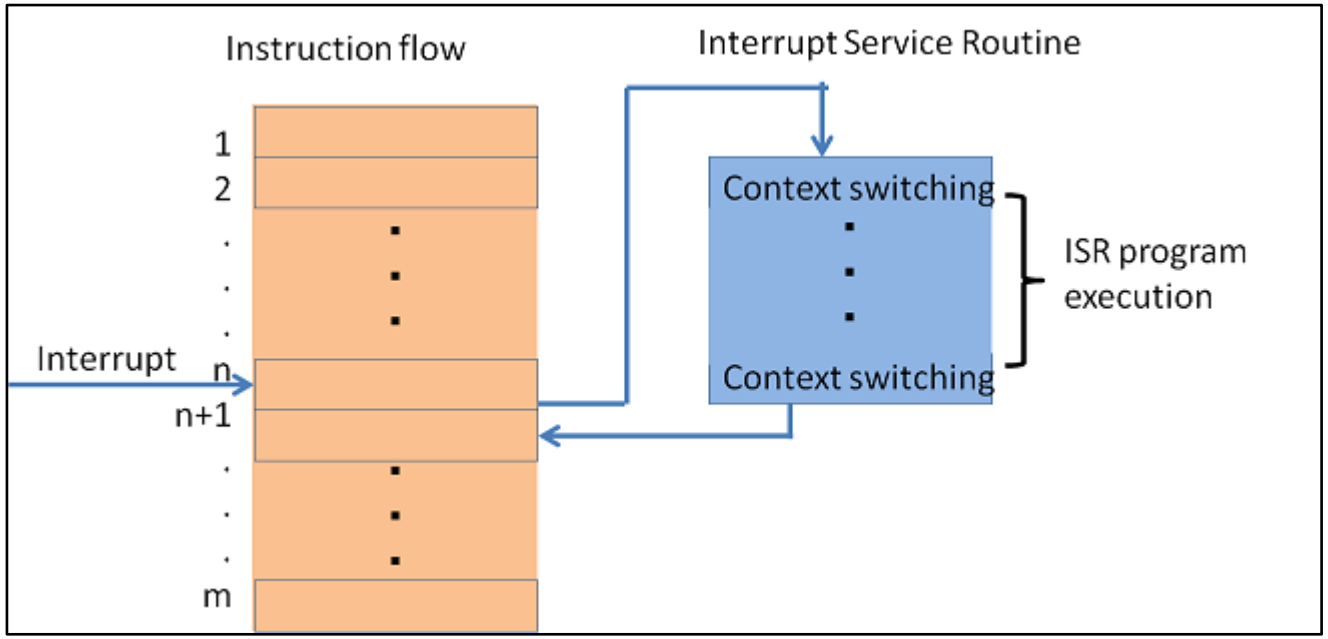
Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- Incoming interrupts are ***disabled*** while another interrupt is being processed to prevent a ***lost interrupt***
- A *trap* is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**

ACHARYA
BENGALURU, INDIA
Click to Edit

Common Functions of Interrupts





ACHARYA
BENGALURU, INDIA

Click to Edit

Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter

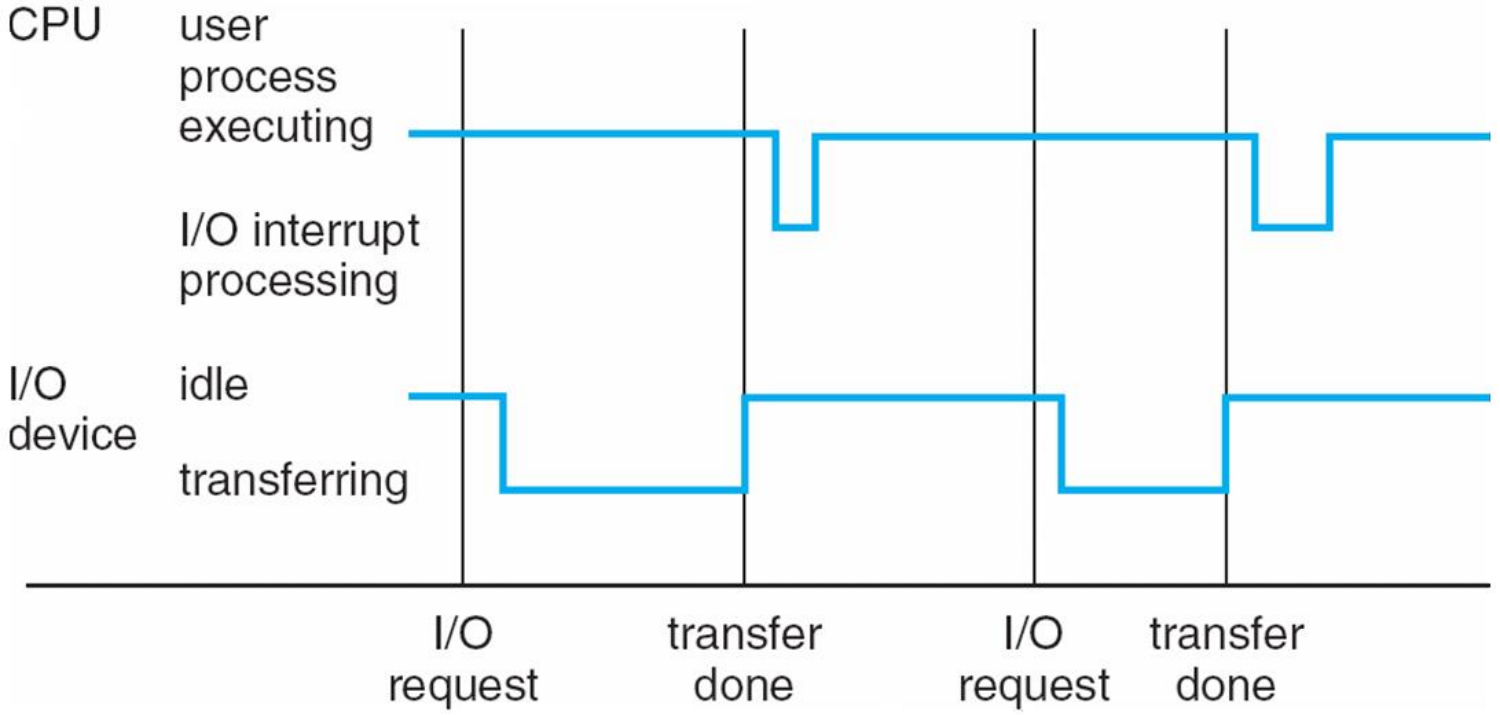


Click to Edit

Interrupt Timeline

ACHARYA
BENGALURU, INDIA

Click to Edit



Storage Structure

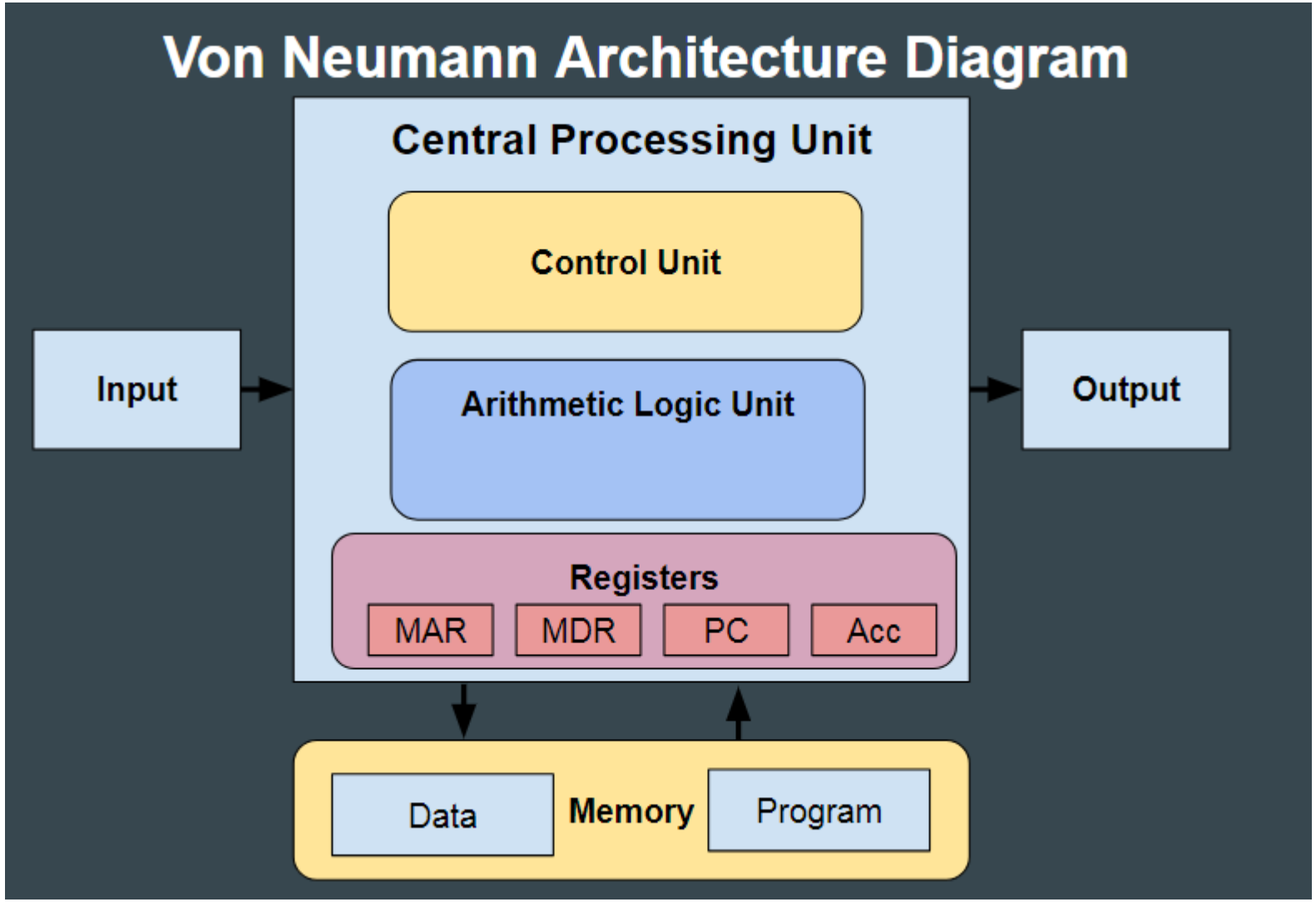
- **Main memory** –
  - Programs must be in the main memory(RAM) to be executed.
  - only large storage media that the CPU can access directly
  - **Volatile**
  - An array of memory words, each word has its own address.
  - **Load and store instruction**
  - Load moves word from main memory to CPU and store instruction moves the content from CPU to main memory.
  - Von Neumann Architecture

ACHARYA
BENGALURU, INDIA
Click to Edit

Storage Structure

ACHARYA
BENGALURU, INDIA

Click to Edit



Storage Structure

- **Secondary storage** – extension of main memory that provides large **nonvolatile** storage capacity

- **Magnetic disks** – stores program and data.
  - Most of the programs are stored until loaded into memory
  - Used as source and destination of the information, so management of disk is of central importance.

ACHARYA
BENGALURU, INDIA

Click to Edit

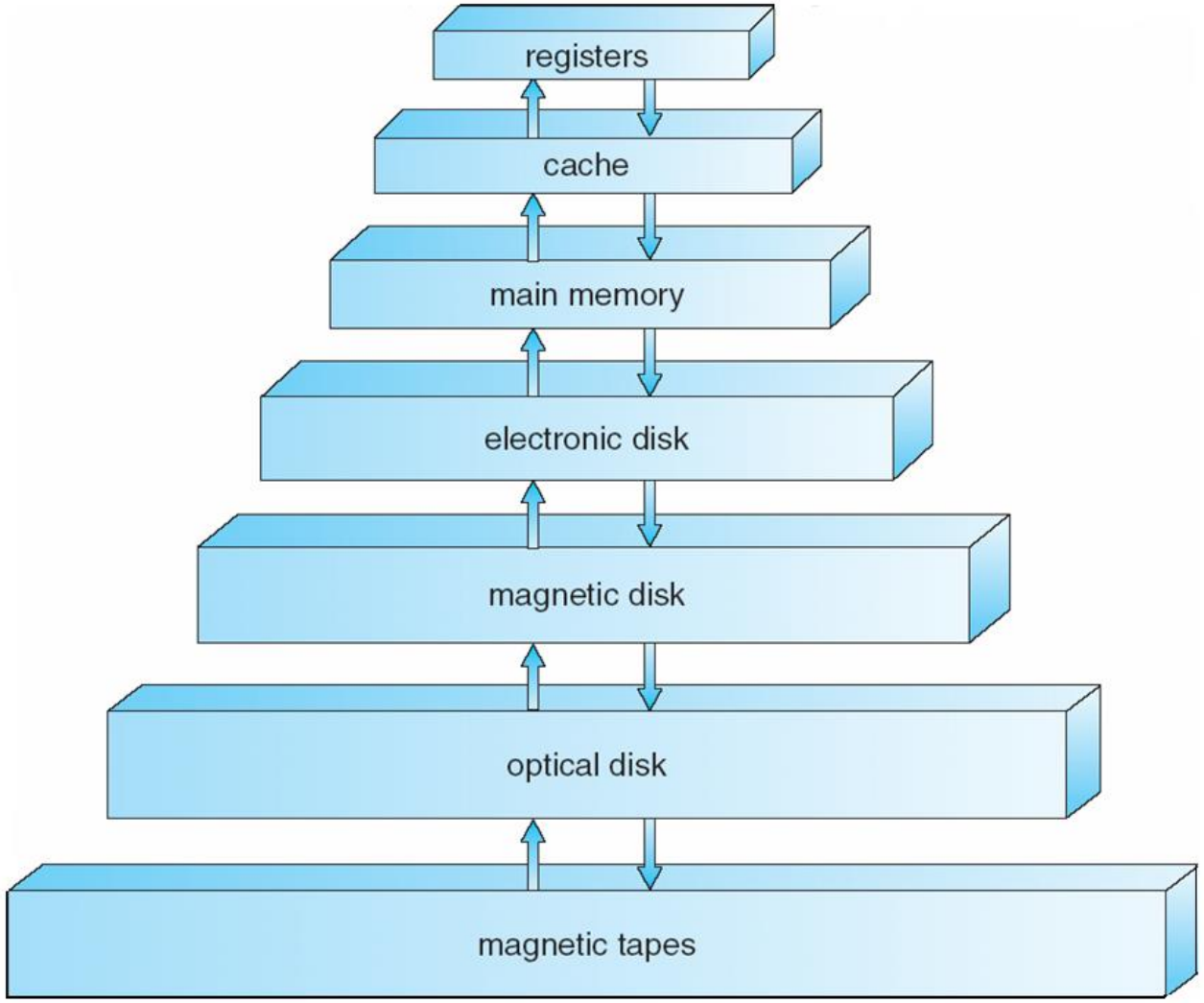
Storage Hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility

- **Caching** – copying information into faster storage system; main memory can be viewed as a *cache* for secondary storage

ACHARYA
BENGALURU, INDIA
Click to Edit

Storage-Device Hierarchy



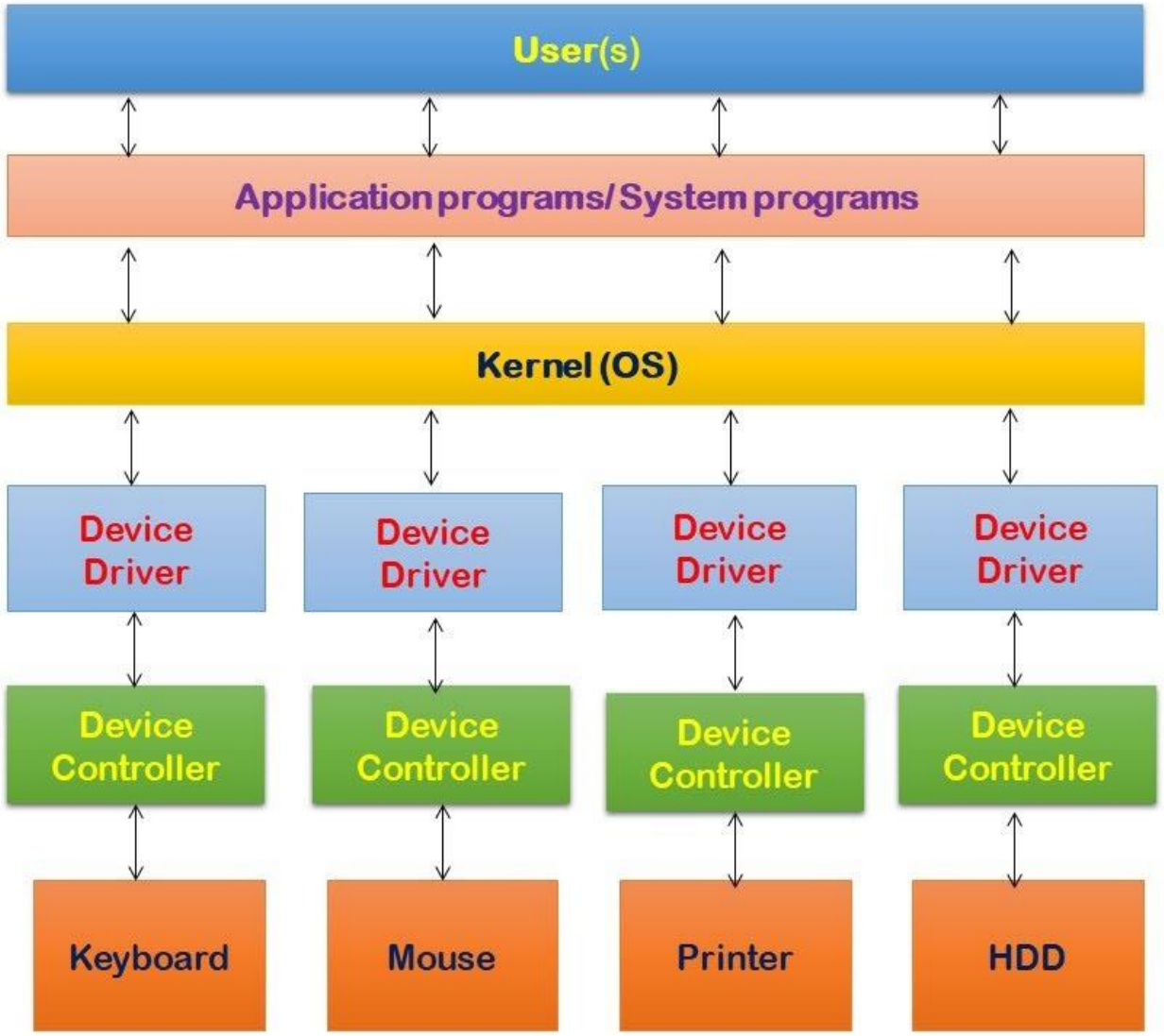
ACHARYA
BENGALURU, INDIA

Click to Edit

I/O Structure

ACHARYA
BENGALURU, INDIA

Click to Edit



Direct Memory Access Structure

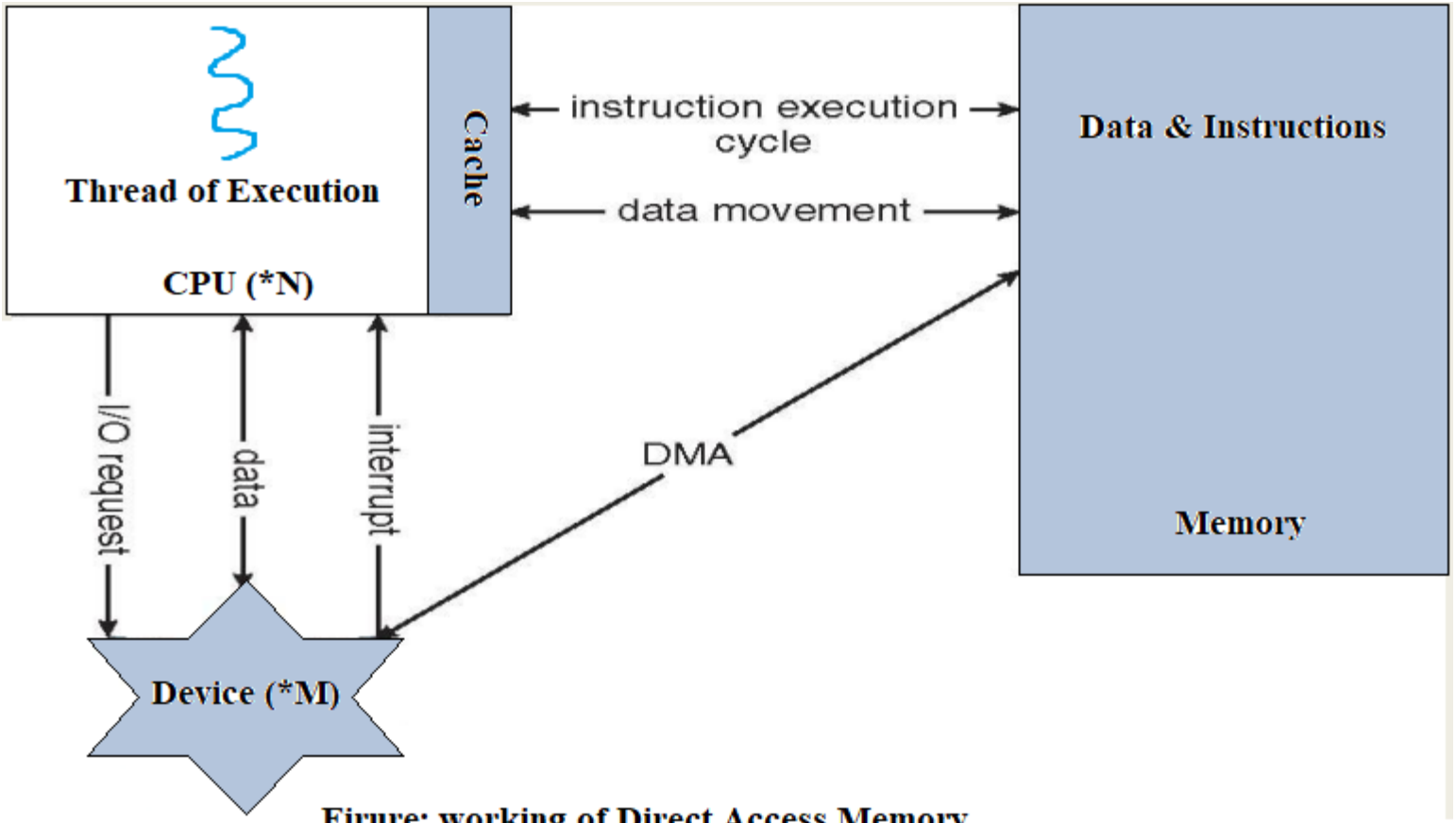


- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte

Direct Memory Access Structure

ACHARYA
BENGALURU, INDIA

Click to Edit



Firure: working of Direct Access Memory

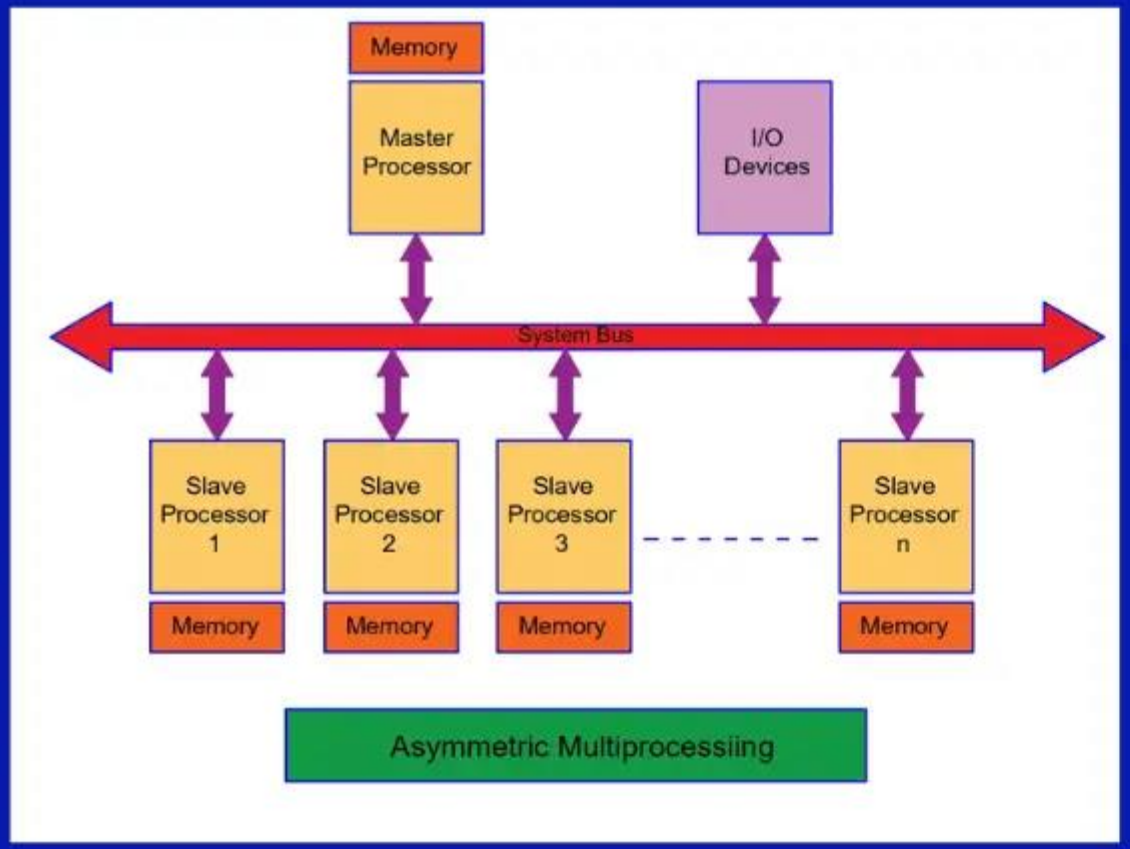
Computer-System Architecture

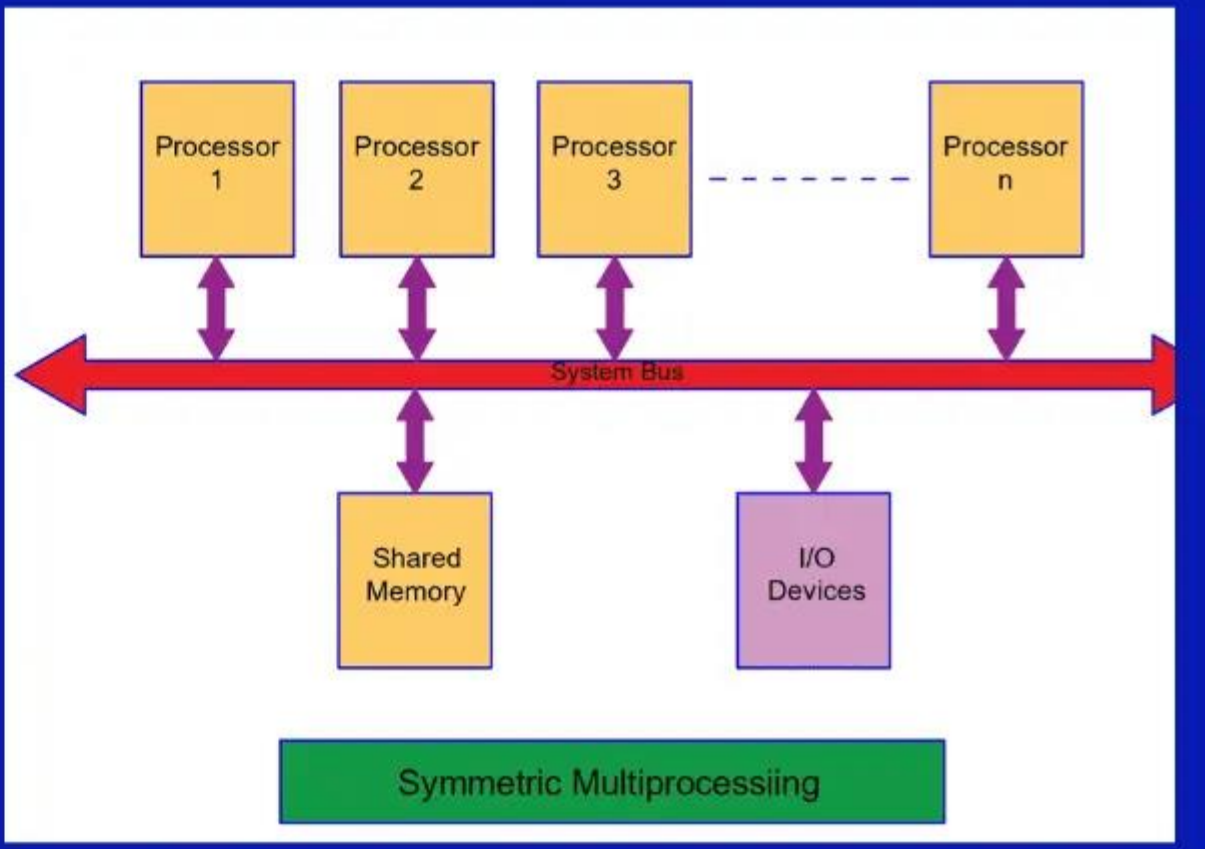


- Most systems use a single general-purpose processor (PDAs through mainframes)
  - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability – graceful degradation** or **fault tolerance**
  - Two types:
    1. **Asymmetric Multiprocessing**
    2. **Symmetric Multiprocessing**

C

SYMMETRIC VS ASYMMETRIC MULTIPROCESSING





Clustered Systems

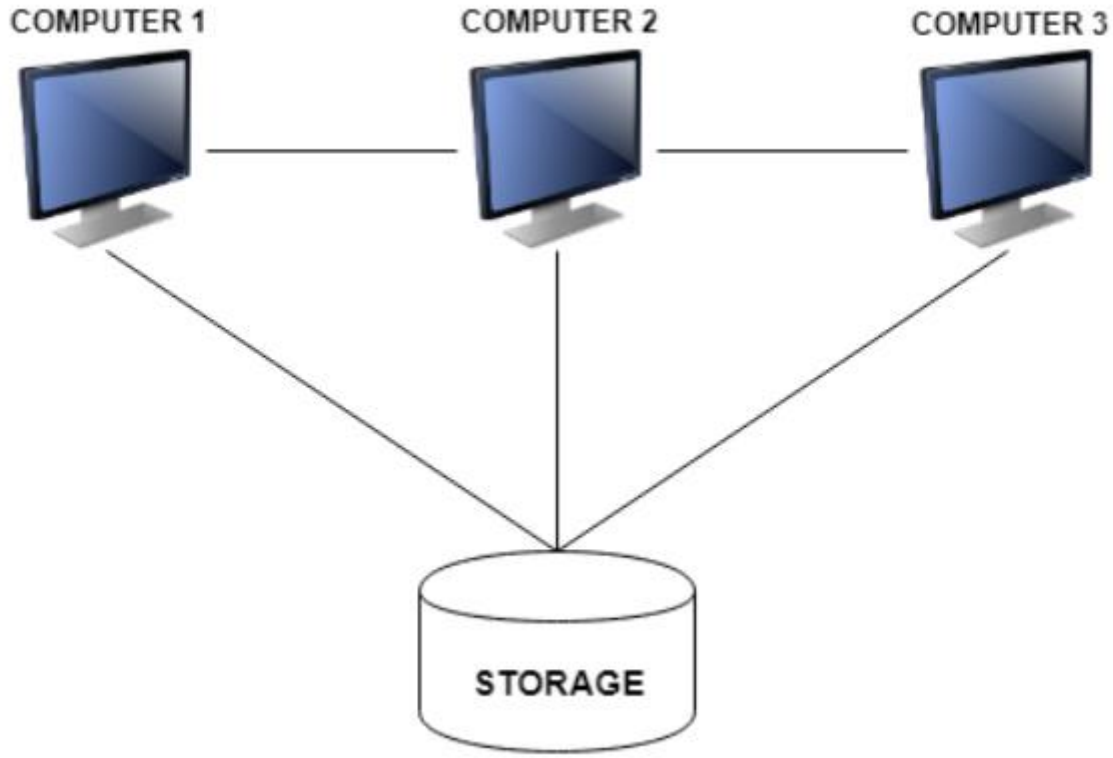


- Like multiprocessor systems, but multiple systems working together
- clustered systems are created by two or more individual computer systems merged together.
  - LAN
  - Usually sharing storage via a **storage-area network (SAN)**
  - Provides a **high-availability** service which survives failures
    - **Asymmetric clustering** has one machine in hot-standby mode
    - **Symmetric clustering** has multiple nodes running applications, monitoring each other
  - Some clusters are for **high-performance computing (HPC)**
    - Applications must be written to use **parallelization**

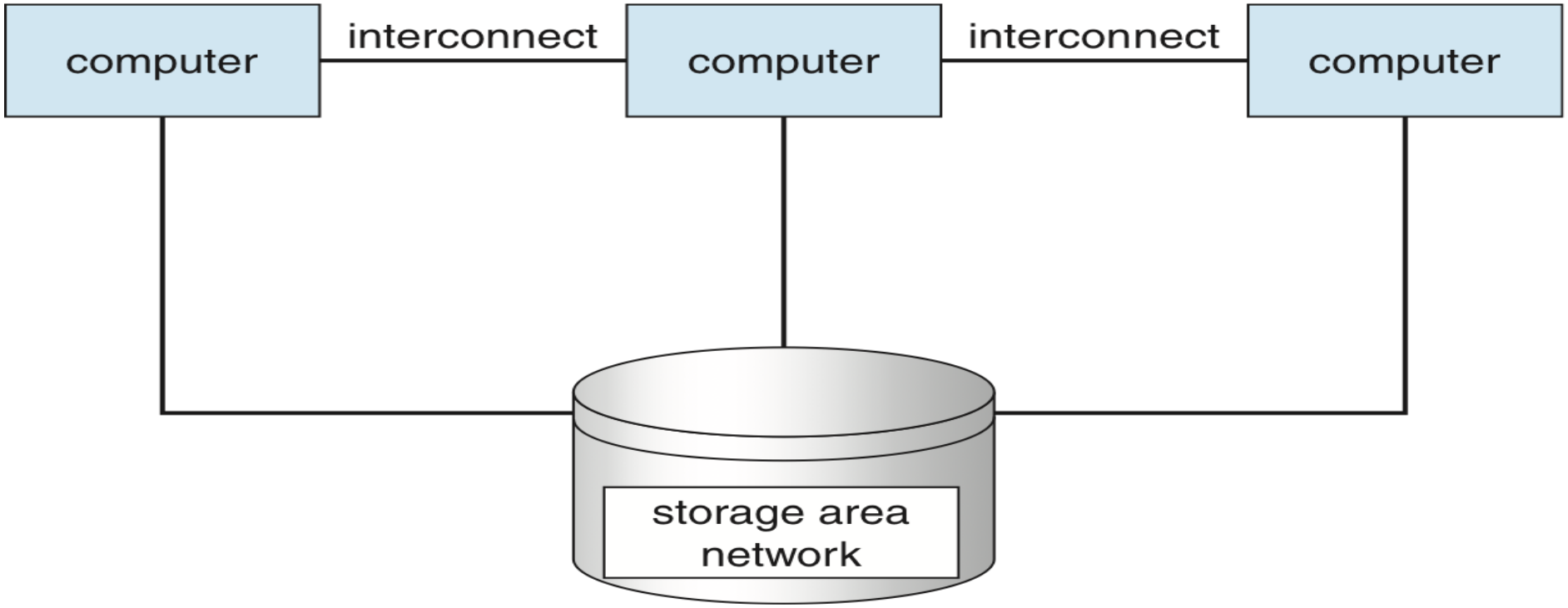
Clustered Systems

ACHARYA
BENGALURU, INDIA

Click to Edit



Clustered Systems



ACHARYA
BENGALURU, INDIA

Click to Edit

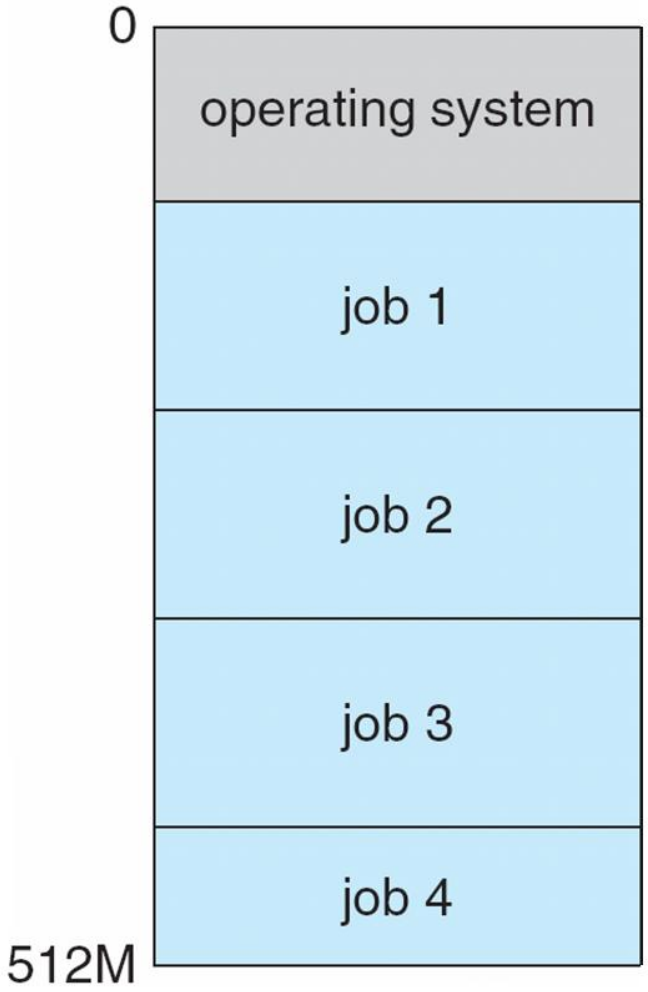
Operating System Structure



Click to Edit

- **Multiprogramming**
- needed for efficiency
- Multiprogramming increases **CPU utilization** by organizing jobs (code and data) so that the CPU always has one to execute.
  - Single user cannot keep CPU and I/O devices busy at all times
  - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
  - multiprogramming require several jobs to be kept simultaneously in memory.
  - the jobs are kept initially on the disk in the **job pool**.
  - A subset of total jobs in system is kept in memory
  - One job selected and run via **job scheduling**
  - When it has to wait (for I/O for example), OS switches to another job

Memory Layout for Multiprogrammed System



ACHARYA
BENGALURU, INDIA

Click to Edit

Operating System Structure



- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory ⇨ **process**
  - If several jobs ready to run at the same time ⇨ **CPU scheduling**
  - If processes don't fit in memory, **swapping** moves them in and out to run

Operating-System Operations

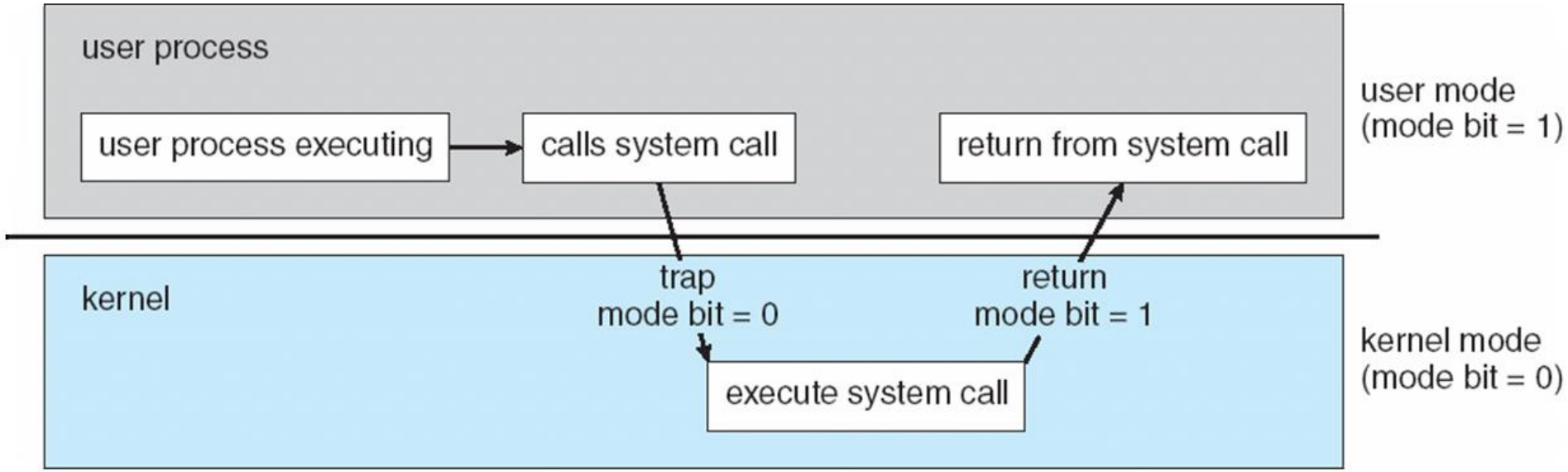
- Interrupt driven by hardware
- Software error or request creates **exception** or **trap**
  - Division by zero, request for operating system service



Click to Edit

Operating-System Operations

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**(also called supervisor mode, system mode, or privileged mode).
  - A bit, called the **mode bit** is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).



ACHARYA
BENGALURU, INDIA

Click to Edit

Operating-System Operations

- The dual mode of operation provides us with the means for **protecting the operating system** from errant users-and errant users from one another.
- privileged instructions

ACHARYA
BENGALURU, INDIA

Click to Edit

Timer

- Ensure that the operating system maintains control over the CPU
- Timer to prevent infinite loop / process hogging resources
  - A timer can be set to interrupt the computer after a specified period.
  - Set interrupt after specific period
  - Operating system decrements counter
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time

ACHARYA
BENGALURU, INDIA

Click to Edit

Process Management

- A process is a program in execution.
- **Program** is a ***passive entity***, **process** is an ***active entity***.
- **Process needs resources** to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- **Process termination** requires reclaim of any reusable resources

ACHARYA
BENGALURU, INDIA

Click to Edit

Process Management

- **Thread** is the segment of a process which means a process can have multiple thread.
- **Single-threaded process** has one **program counter** specifying location of next instruction to execute
  - Process executes instructions sequentially, one at a time, until completion
- **Multi-threaded process** has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs

ACHARYA
BENGALURU, INDIA
Click to Edit

Process Management Activities



The operating system is responsible for the following activities in connection with process management:

- **Creating and deleting** both user and system processes
- **Suspending and resuming** processes
- Providing mechanisms for **process synchronization**
- Providing mechanisms for **process communication**
- Providing mechanisms for **deadlock handling**

Memory Management

- All data in memory before and after processing
- All instructions in memory in order to execute
- Memory management determines what is in memory when
  - Optimizing CPU utilization and computer response to users
- **Memory management activities**
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

ACHARYA
BENGALURU, INDIA
Click to Edit

Storage Management

- OS provides uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - **file**
  - Each medium is controlled by device (i.e., disk drive, tape drive)
    - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- **File-System management**
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

ACHARYA
BENGALURU, INDIA
Click to Edit

Mass-Storage Management

- The computer system must provide **secondary storage** to back up main memory.
- Most modern computer systems use **disks** as the principal on-line storage medium for both programs and data.
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms.

ACHARYA
BENGALURU, INDIA

Click to Edit

Mass-Storage Management

- **OS activities**
  - Free-space management
  - Storage allocation
  - Disk scheduling
- **Some storage need not be fast**
  - Tertiary storage includes optical storage, magnetic tape
  - Still must be managed – by OS or applications
  - Varies between WORM (write-once, read-many-times) and RW (read-write)

ACHARYA
BENGALURU, INDIA
Click to Edit

Caching

- Information is normally kept in some storage system (such as **main memory**).
- As it is used,it is copied into a faster storage system-the **cache-on** a temporary basis.
- In addition, **internal programmable registers**, such as index registers, provide a high-speed cache for main memory.
- **register-allocation and register-replacement algorithms**
- **cache management-** cache size and of a replacement policy

ACHARYA
BENGALURU, INDIA

Click to Edit

Performance of Various Levels of Storage

- Movement between levels of storage hierarchy can be explicit or implicit

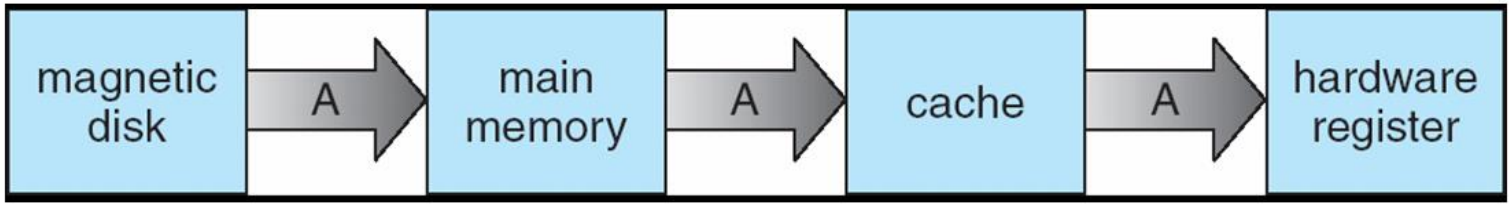
ACHARYA
BENGALURU, INDIA

Click to Edit

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

Migration of Integer A from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache

ACHARYA
BENGALURU, INDIA
Click to Edit

I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user

- I/O subsystem responsible for
  - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
  - General device-driver interface
  - Drivers for specific hardware devices

ACHARYA
BENGALURU, INDIA
Click to Edit

Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control
  - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
  - **Privilege escalation** allows user to change to effective ID with more rights

ACHARYA
BENGALURU, INDIA
Click to Edit

Distributed Computing



Click to Edit

- Collection of separate, possibly heterogeneous, systems networked together
  - Network is a communications path
    - Local Area Network (**LAN**)
    - Wide Area Network (**WAN**)
    - Metropolitan Area Network (**MAN**)
- Network Operating System provides features between systems across the network
  - Communication scheme allows systems to exchange messages
  - Illusion of a single system

Special-Purpose Systems



Click to Edit

- Real-time embedded systems most prevalent form of computers
  - Vary considerable, special purpose, limited purpose OS, **real-time OS**
- Multimedia systems
  - Streams of data must be delivered according to time restrictions
- Handheld systems
  - PDAs, smart phones, limited CPU, memory, power
  - Reduced feature set OS, limited I/O

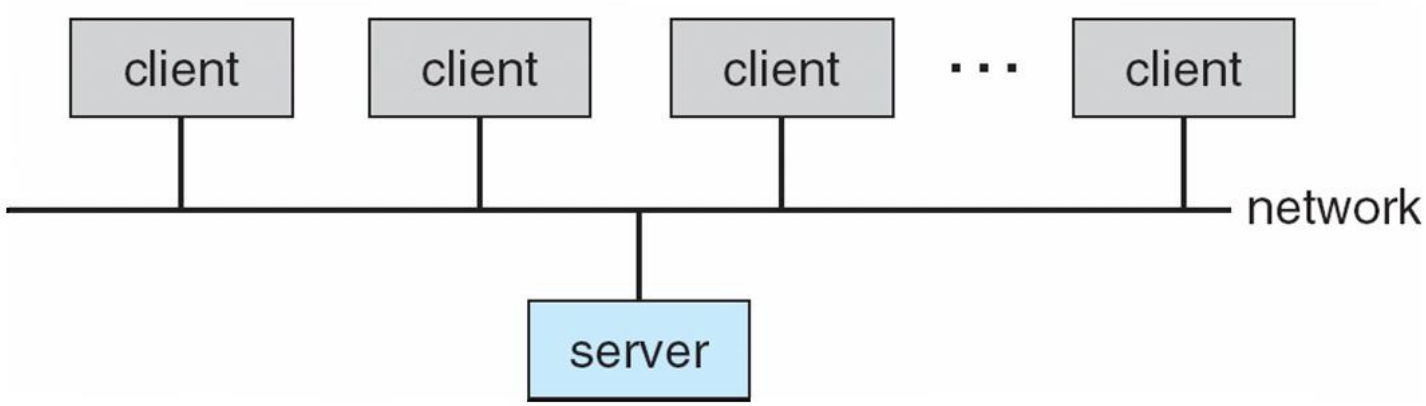
Computing Environments

- Traditional computer
  - Blurring over time
  - Office environment
    - PCs connected to a network, terminals attached to mainframe or minicomputers providing batch and timesharing
    - Now portals allowing networked and remote systems access to same resources
  - Home networks
    - Used to be single system, then modems
    - Now firewalled, networked

ACHARYA
BENGALURU, INDIA
Click to Edit

Computing Environments (Cont.)

- Client-Server Computing
  - Dumb terminals supplanted by smart PCs
  - Many systems now **servers**, responding to requests generated by **clients**
    - **Compute-server** provides an interface to client to request services (i.e., database)
    - **File-server** provides interface for clients to store and retrieve files



ACHARYA
BENGALURU, INDIA

Click to Edit

Peer-to-Peer Computing

- Another model of distributed system

- P2P does not distinguish clients and servers
  - Instead all nodes are considered peers
  - May each act as client, server or both
  - Node must join P2P network
    - Registers its service with central lookup service on network, or
    - Broadcast request for service and respond to requests for service via **discovery protocol**

ACHARYA
BENGALURU, INDIA
Click to Edit

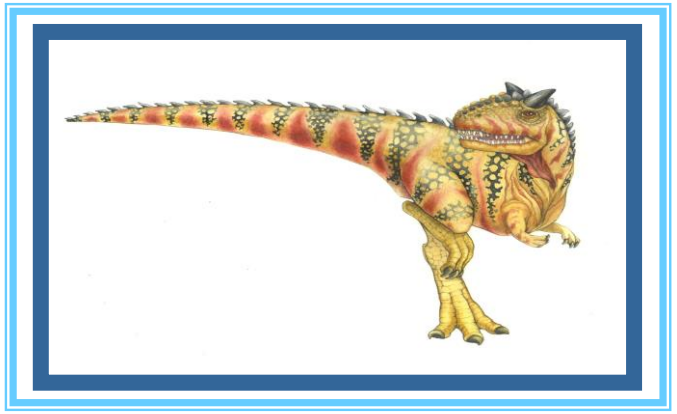
Web-Based Computing



Click to Edit

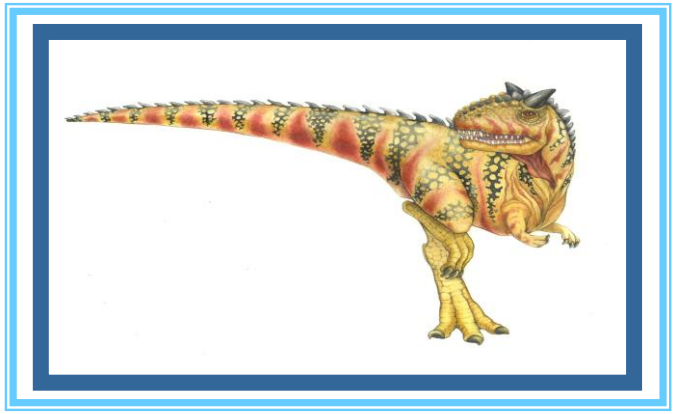
- Web has become ubiquitous
- PCs most prevalent devices
- More devices becoming networked to allow web access
- New category of devices to manage web traffic among similar servers: **load balancers**
- Use of operating systems like Windows 95, client-side, have evolved into Linux and Windows XP, which can be clients and servers

End of Chapter 1



Silberschatz, Galvin and Gagne ©2009

Chapter 2: Operating-System Structures



Silberschatz, Galvin and Gagne ©2009

Chapter 2: Operating-System Structures



Click to Edit

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- Operating System Structure
- Virtual Machines
- Operating System Debugging
- Operating System Generation
- System Boot

Objectives

- To describe the services an operating system provides to users, processes, and other systems
- To discuss the various ways of structuring an operating system
- To explain how operating systems are installed and customized and how they boot

ACHARYA
BENGALURU, INDIA

Click to Edit

Operating System Services



Click to Edit

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (UI).
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

Operating System Services (Cont.)

- **Communications** – Processes may exchange information, on the same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by the OS)
- **Error detection** – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

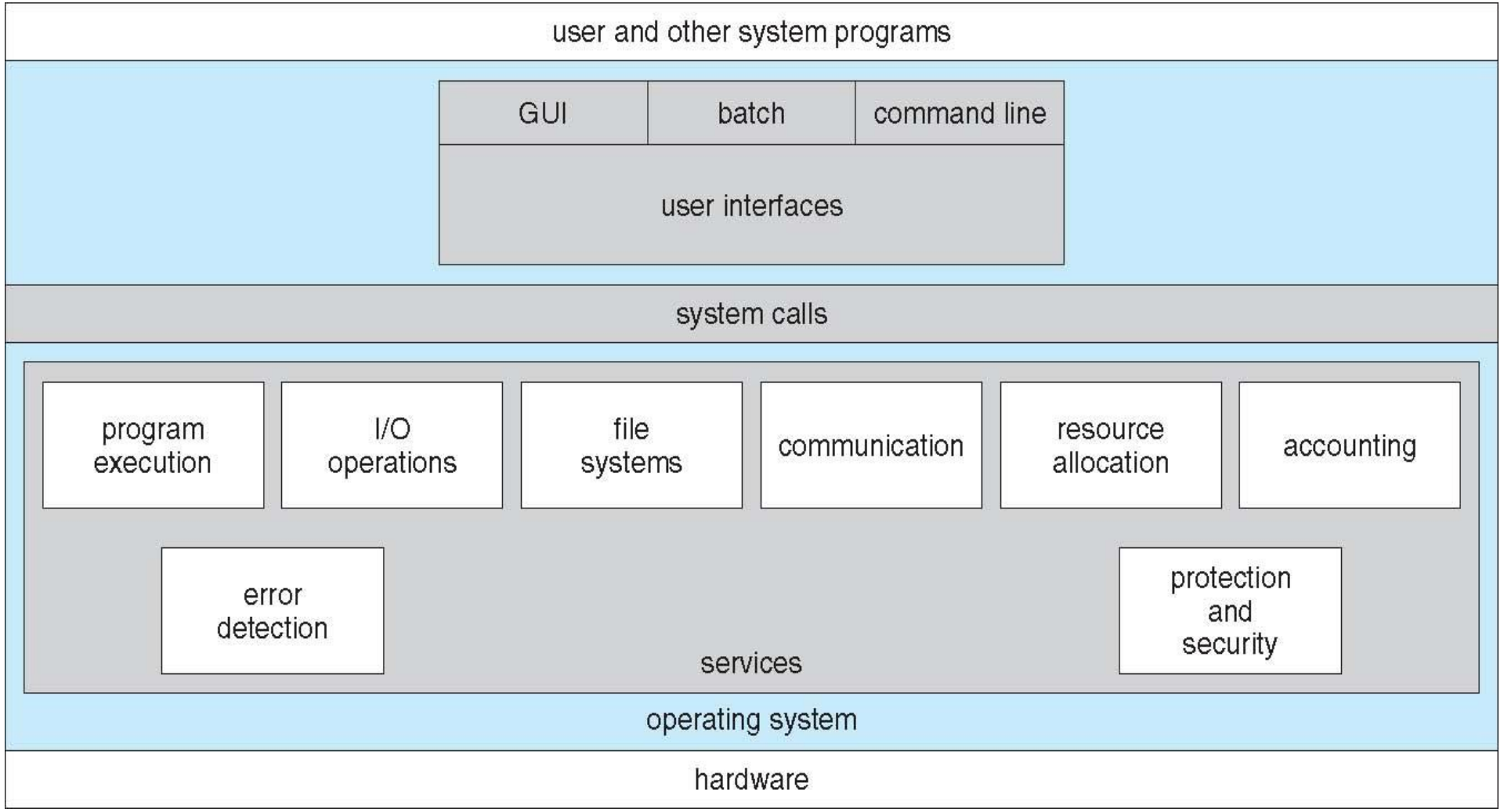
ACHARYA
BENGALURU, INDIA
Click to Edit

Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
    - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.

ACHARYA
BENGALURU, INDIA
Click to Edit

A View of Operating System Services



ACHARYA
BENGALURU, INDIA

Click to Edit

User Operating System Interface - CLI

- Command Line Interface (CLI) or **command interpreter** allows direct command entry
  - Sometimes implemented in kernel, sometimes by systems program
  - Sometimes multiple flavors implemented – **shells**
  - Primarily fetches a command from user and executes it

ACHARYA
BENGALURU, INDIA

Click to Edit

User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI "command" shell
  - Apple Mac OS X as "Aqua" GUI interface with UNIX kernel underneath and shells available
  - Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)

ACHARYA
BENGALURU, INDIA
Click to Edit

System Calls

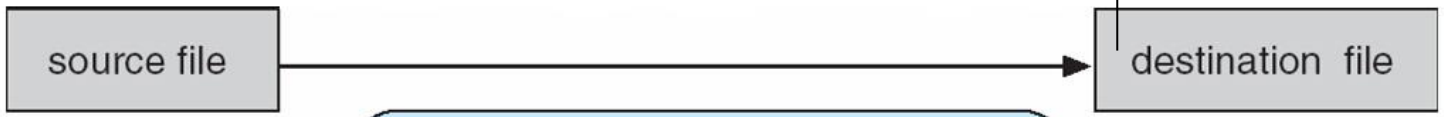
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Program Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?

  (Note that the system-call names used throughout this text are generic)

ACHARYA
BENGALURU, INDIA
Click to Edit

Example of System Calls

- System call sequence to copy the contents of one file to another file



Example System Call Sequence

```
Acquire input file name
  Write prompt to screen
  Accept input
Acquire output file name
  Write prompt to screen
  Accept input
Open the input file
  if file doesn't exist, abort
Create output file
  if file exists, abort
Loop
  Read from input file
  Write to output file
Until read fails
Close output file
Write completion message to screen
Terminate normally
```

ACHARYA
BENGALURU, INDIA

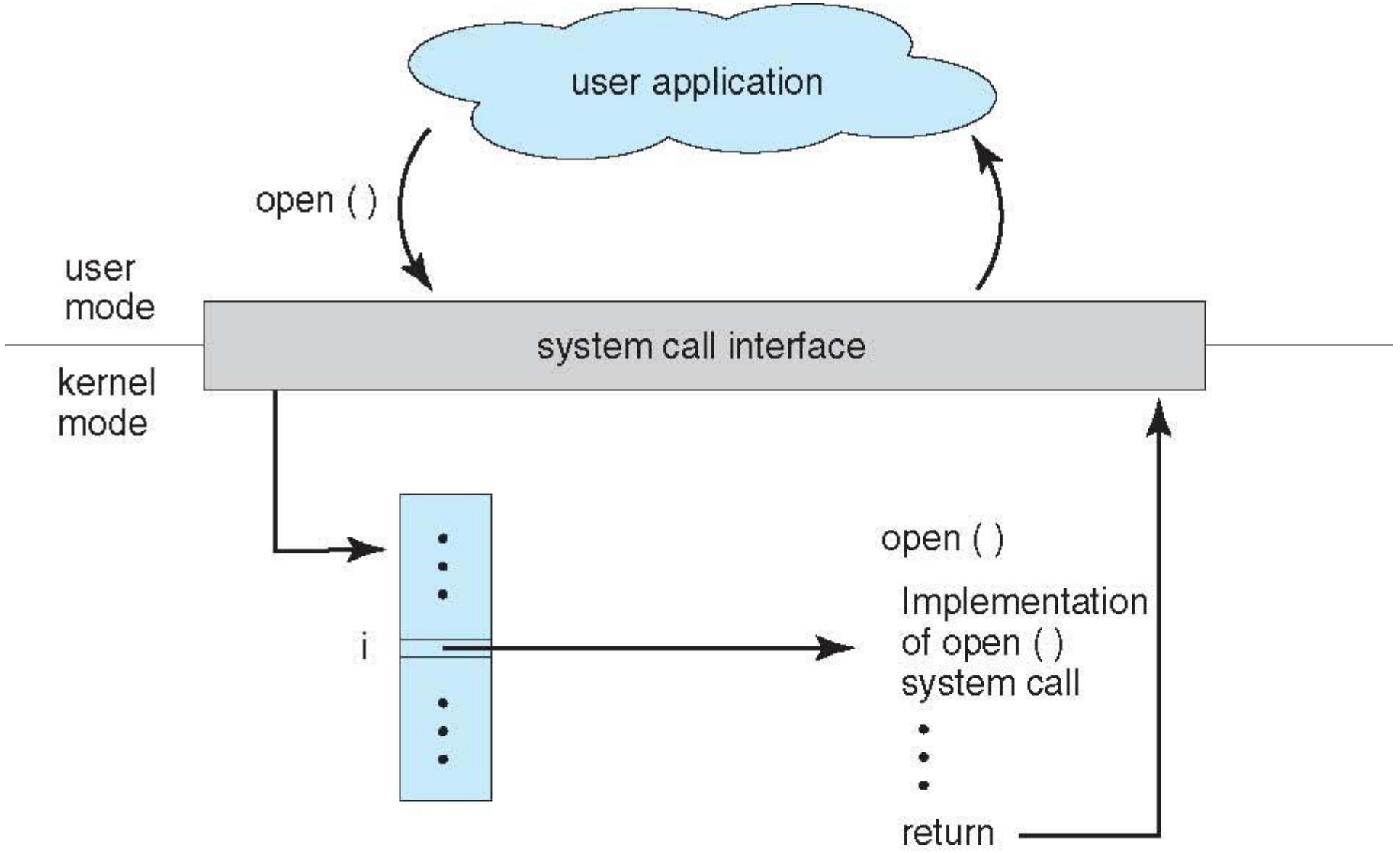
Click to Edit

System Call Implementation

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

ACHARYA
BENGALURU, INDIA
Click to Edit

API – System Call – OS Relationship

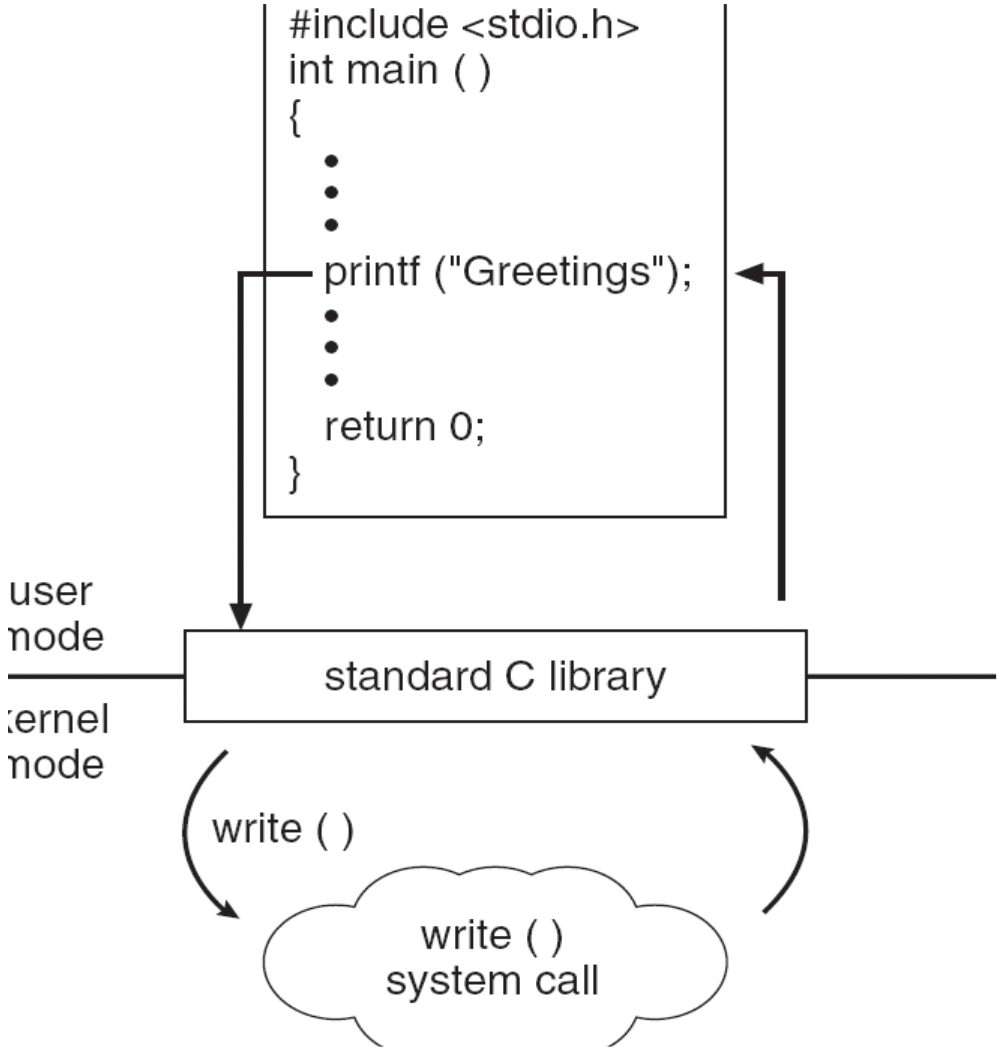


ACHARYA
BENGALURU, INDIA

Click to Edit

Standard C Library Example

- C program invoking printf() library call, which calls write() system call



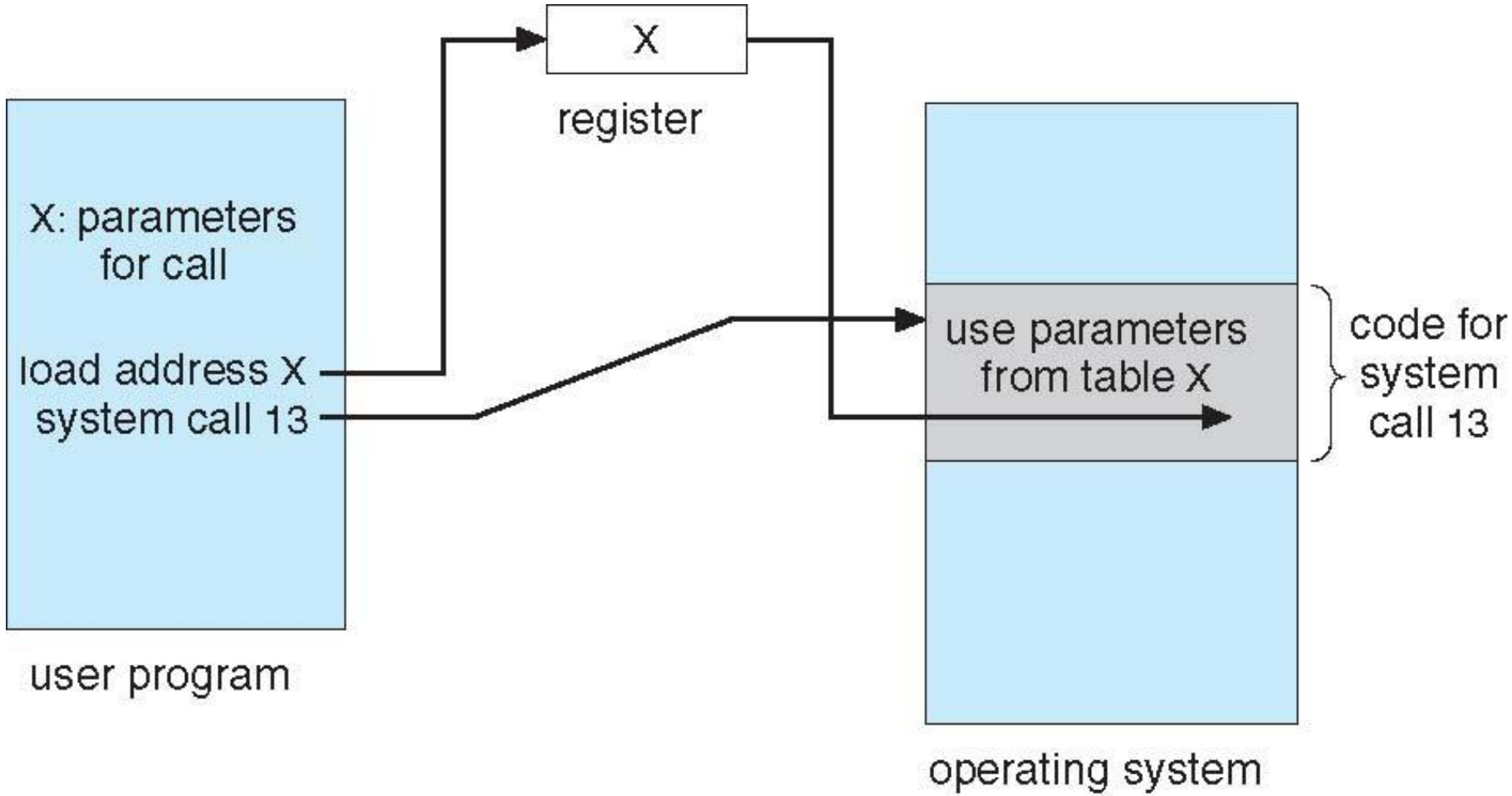
ACHARYA
BENGALURU, INDIA
Click to Edit

System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in *registers*
    - In some cases, may be more parameters than registers
  - Parameters stored in a *block,* or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or *pushed,* onto the *stack* by the program and *popped* off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

ACHARYA
BENGALURU, INDIA
Click to Edit

Parameter Passing via Table



ACHARYA
BENGALURU, INDIA

Click to Edit

System Programs

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a registry - used to store and retrieve configuration information

ACHARYA
BENGALURU, INDIA
Click to Edit

System Programs (Cont.)



Click to Edit

- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

Operating System Design and Implementation



Click to Edit

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start by defining goals and specifications
- Affected by choice of hardware, type of system
- *User* goals and *System* goals
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

Operating System Design and Implementation (Cont.)



- Important principle to separate

  Policy: What will be done?
  Mechanism: How to do it?

- Mechanisms determine how to do something, policies decide what will be done
  - The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later

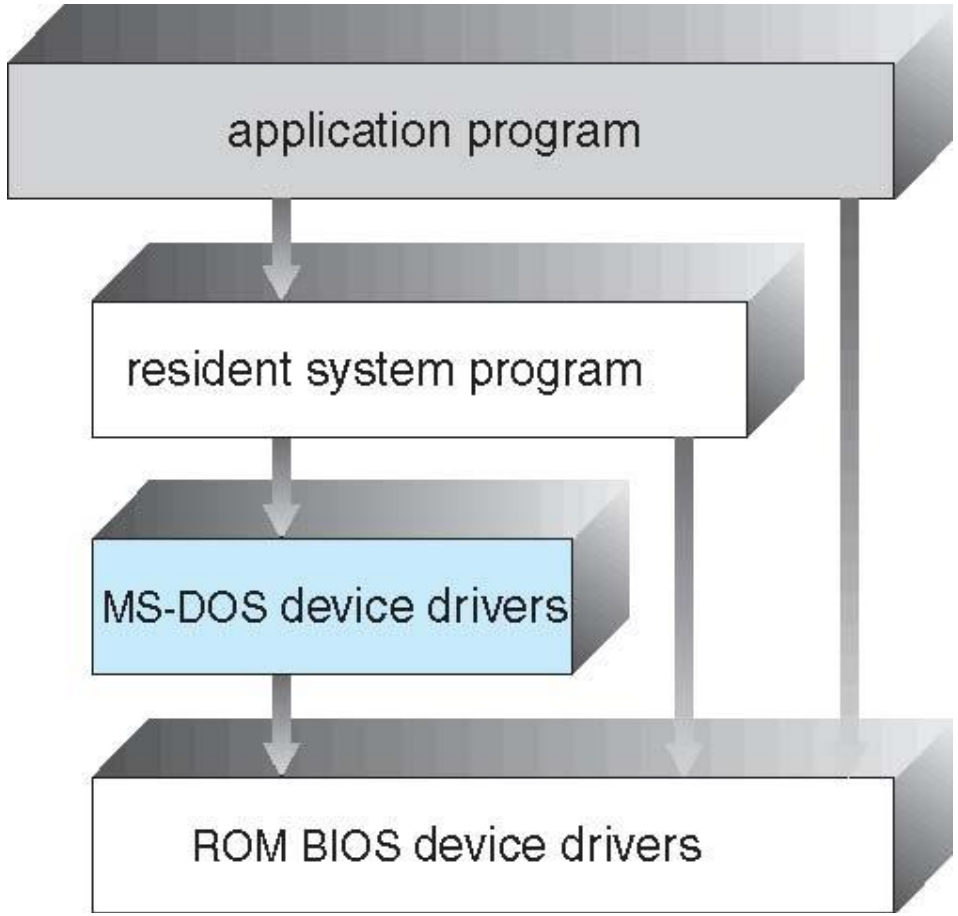
Simple Structure

- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

ACHARYA
BENGALURU, INDIA

Click to Edit

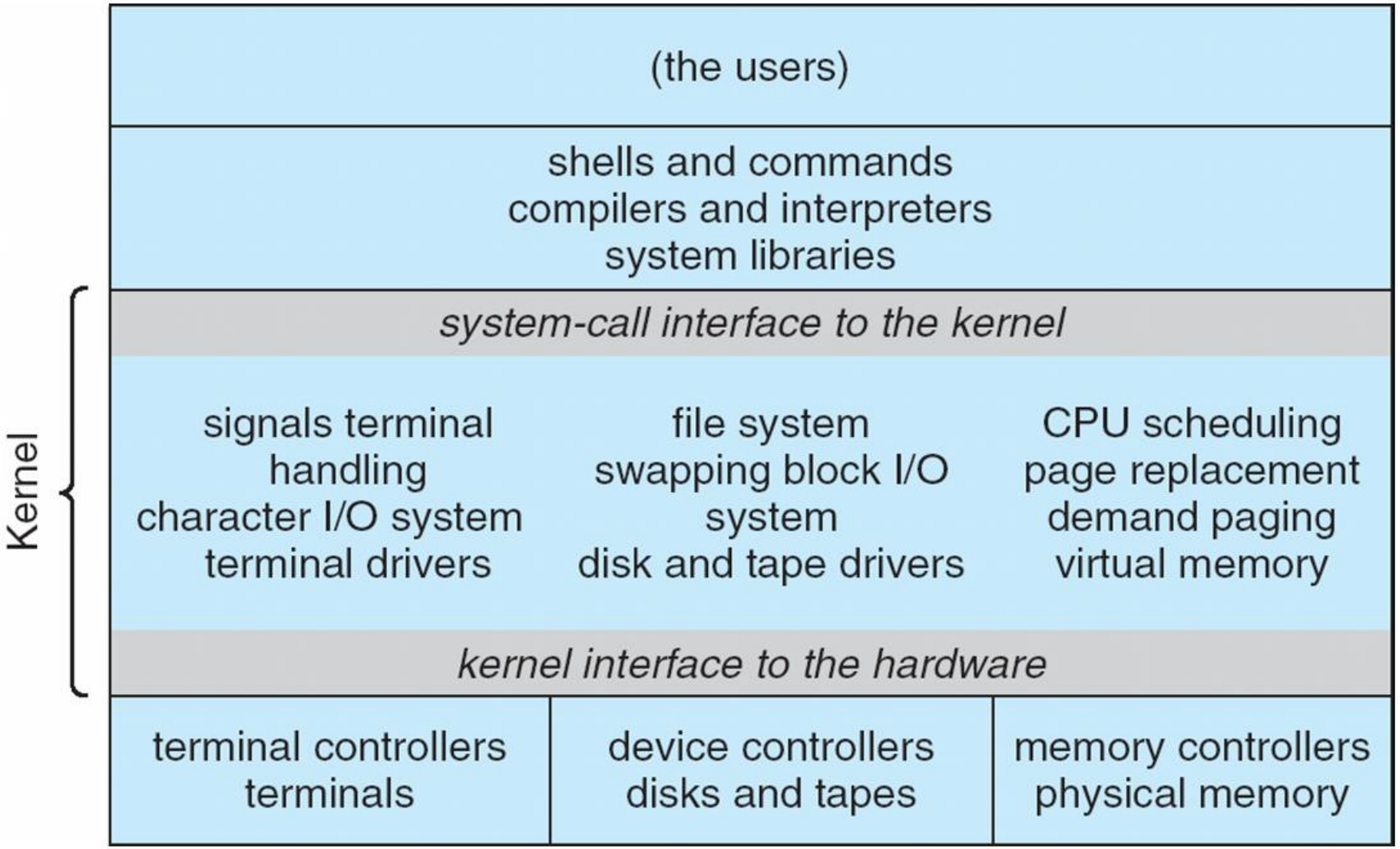
MS-DOS Layer Structure



ACHARYA
BENGALURU, INDIA

Click to Edit

Traditional UNIX System Structure



ACHARYA
BENGALURU, INDIA

Click to Edit

UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts
  - **Systems programs**
  - **The kernel**
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

ACHARYA
BENGALURU, INDIA
Click to Edit

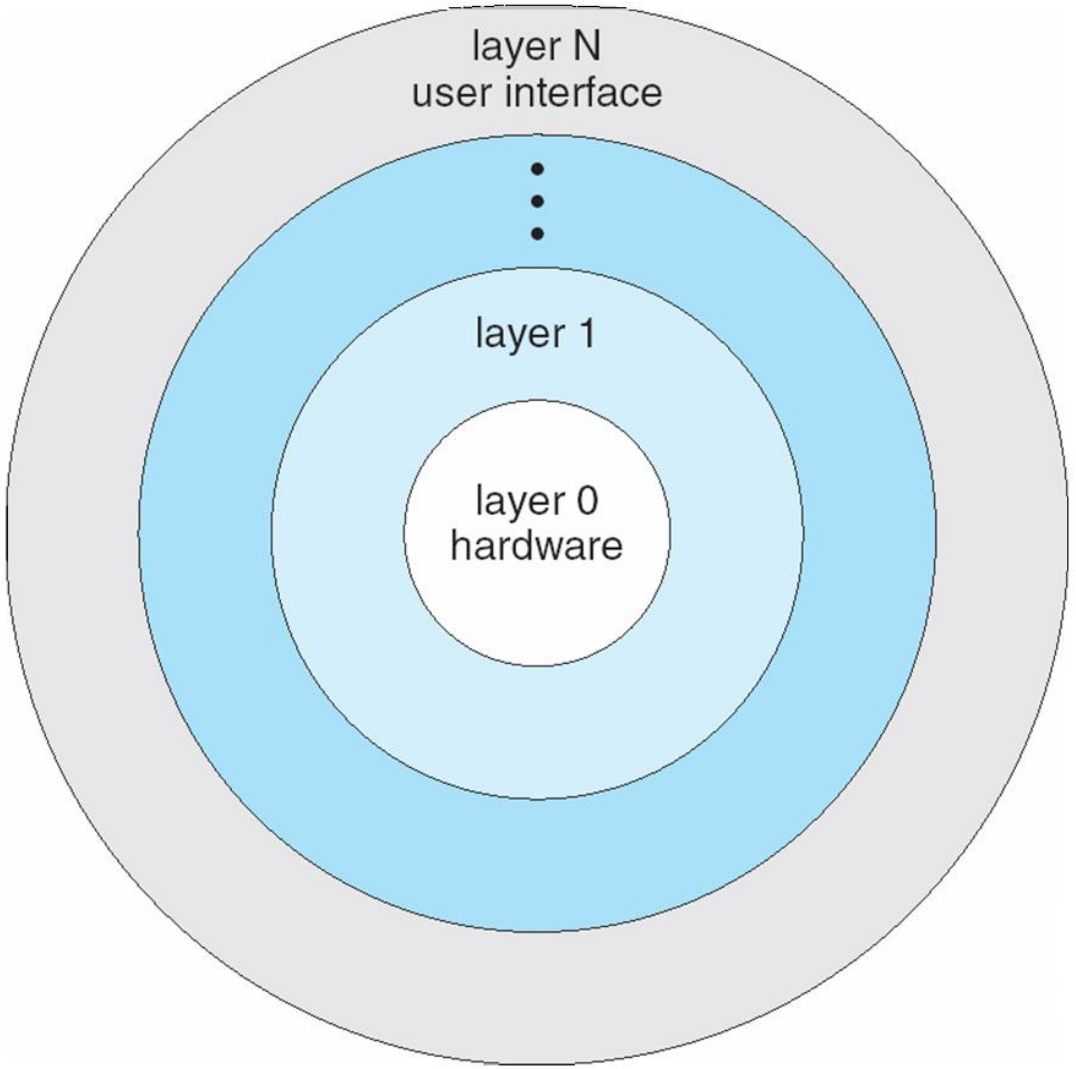
Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers.
- The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.

- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

ACHARYA
BENGALURU, INDIA
Click to Edit

Layered Operating System



ACHARYA
BENGALURU, INDIA

Click to Edit

Microkernel System Structure

- Moves as much from the kernel into "*user*" space
- Communication takes place between user modules using message passing
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

ACHARYA
BENGALURU, INDIA
Click to Edit

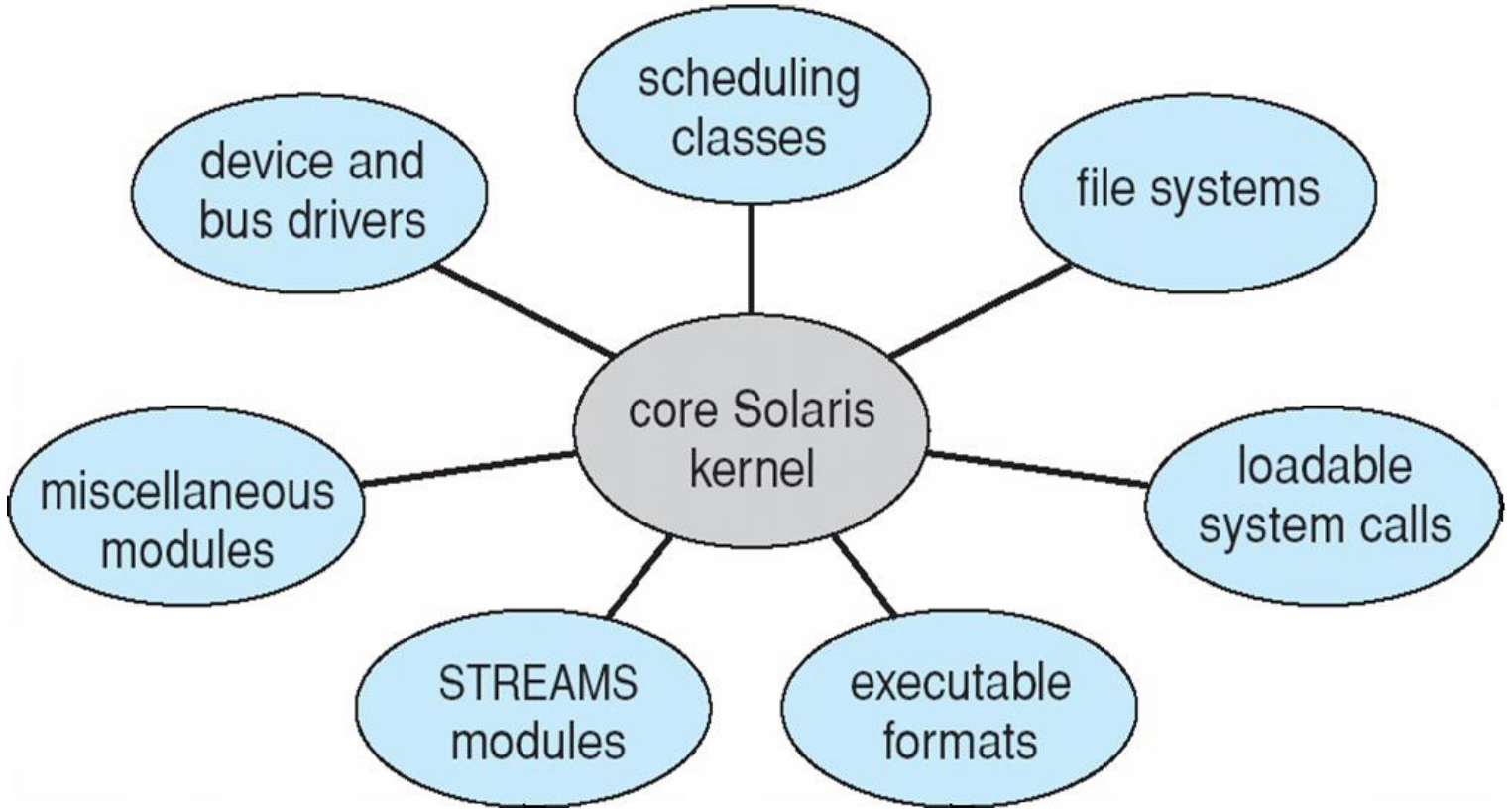
Modules



Click to Edit

- Most modern operating systems implement kernel modules
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible

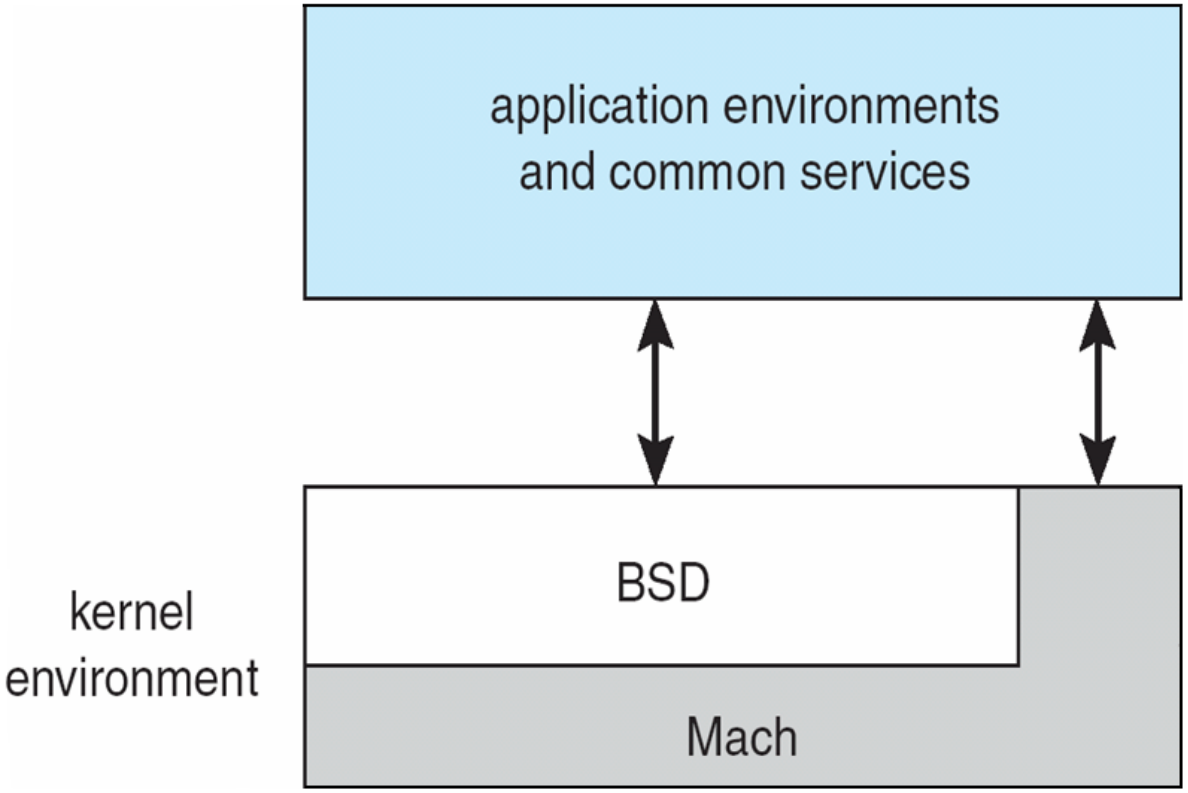
Solaris Modular Approach



ACHARYA
BENGALURU, INDIA

Click to Edit

Mac OS X Structure



ACHARYA
BENGALURU, INDIA

Click to Edit

Virtual Machines

- The layered approach described in Section 2.7.2 is taken to its logical conclusion in the concept of a **virtual machine**.
- The fundamental idea behind a virtual machine is to abstract the hardware of a single computer (the CPU, memory, disk drives, network interface cards, and so forth) into several different execution environments, thereby creating the illusion that each separate execution environment is running its own private computer.
- By using **CPU scheduling and virtual-memory techniques**, an operating system can create the illusion that a process has its own processor with its own **(virtual) memory.**

ACHARYA
BENGALURU, INDIA
Click to Edit

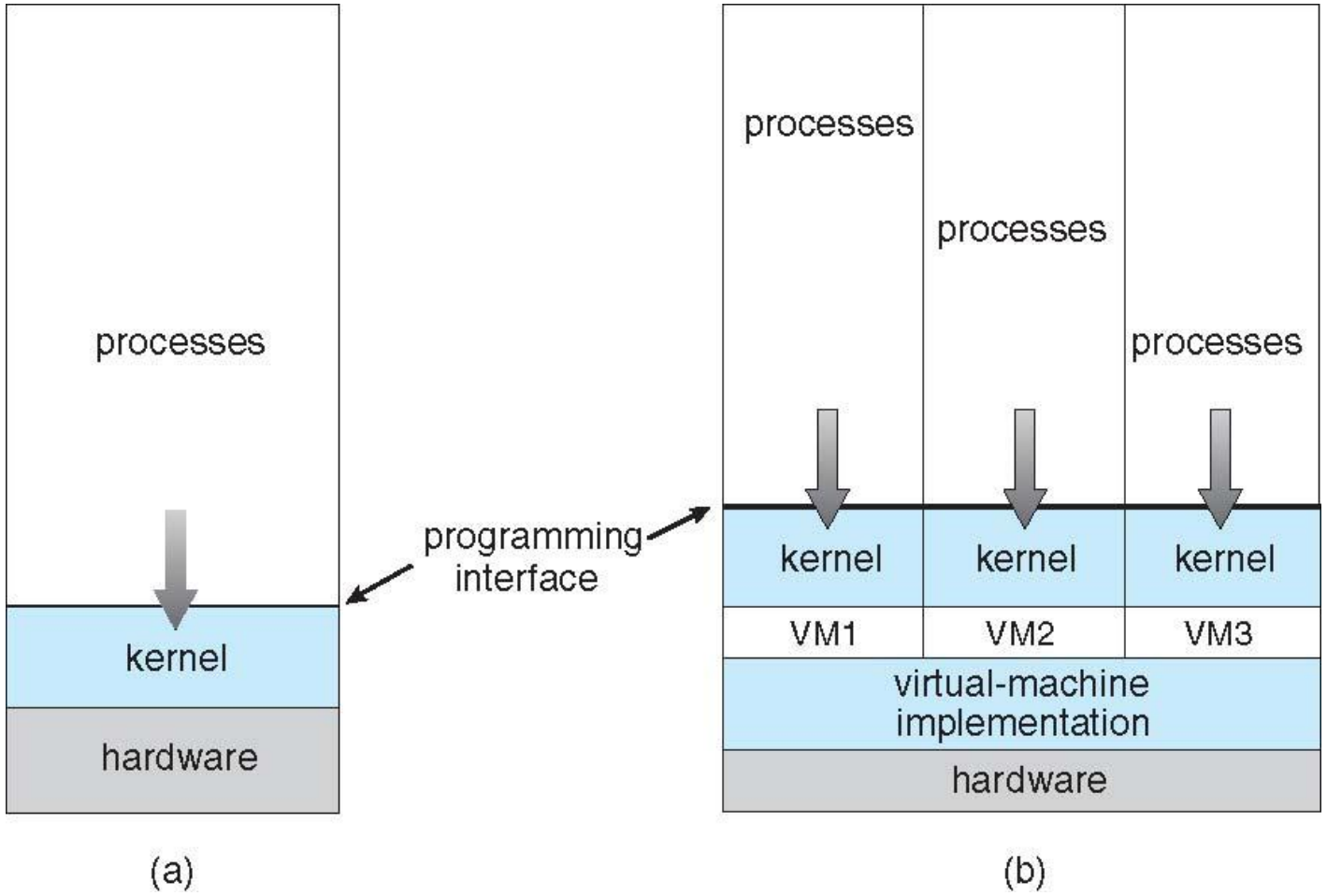
Virtual Machines History and Benefits



Click to Edit

- First appeared commercially in IBM mainframes in 1972
- Fundamentally, multiple execution environments (different operating systems) can share the same hardware
- Protect from each other
- Some sharing of file can be permitted, controlled communicate with each other, other physical systems via networking
- Useful for development, testing **Consolidation** of many low-resource use systems onto fewer busier systems
- "Open Virtual Machine Format", standard format of virtual machines, allows a VM to run within many different virtual machine (host) platforms.

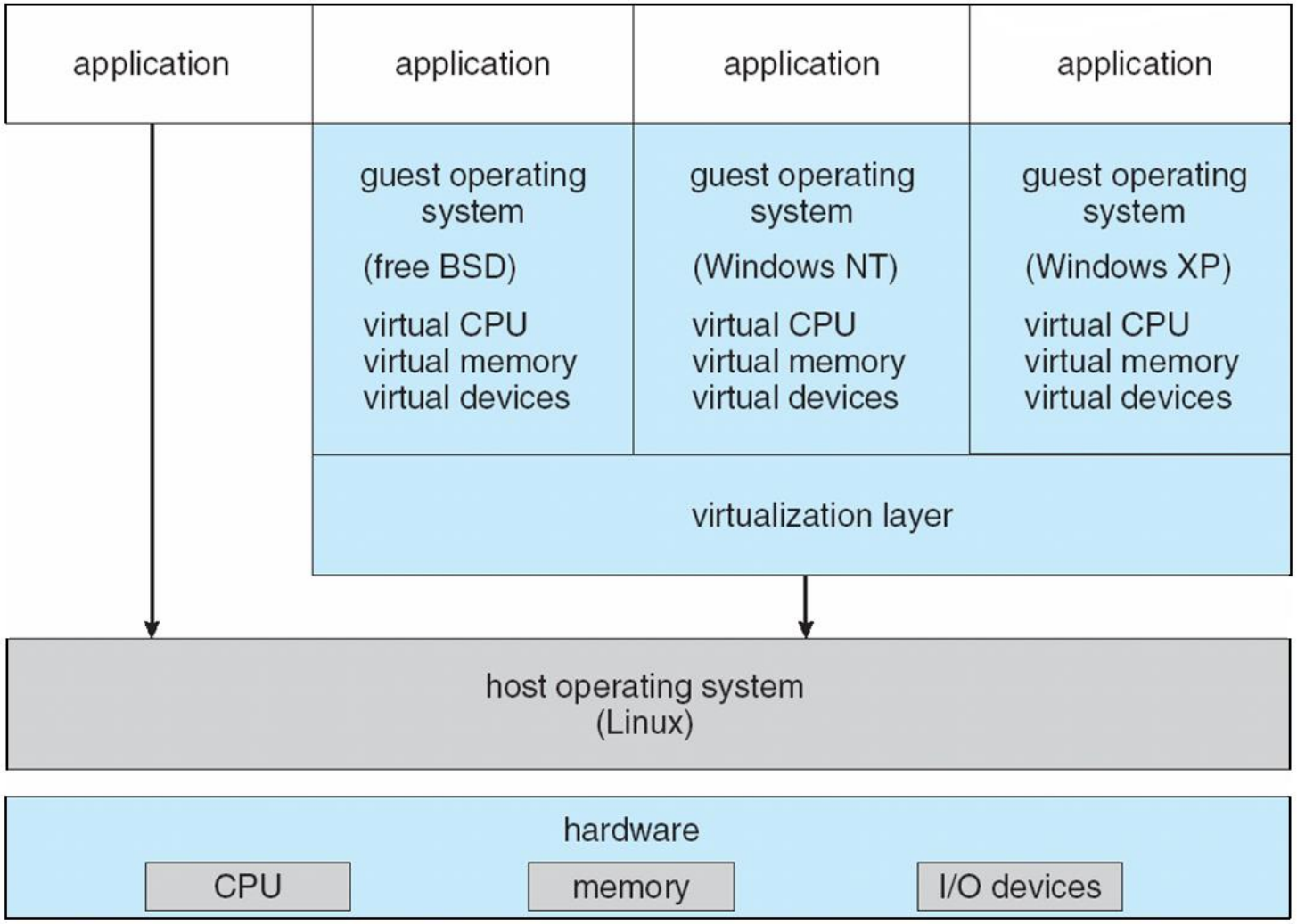
Virtual Machines (Cont.)



(a) Nonvirtual machine (b) virtual machine

Click to Edit

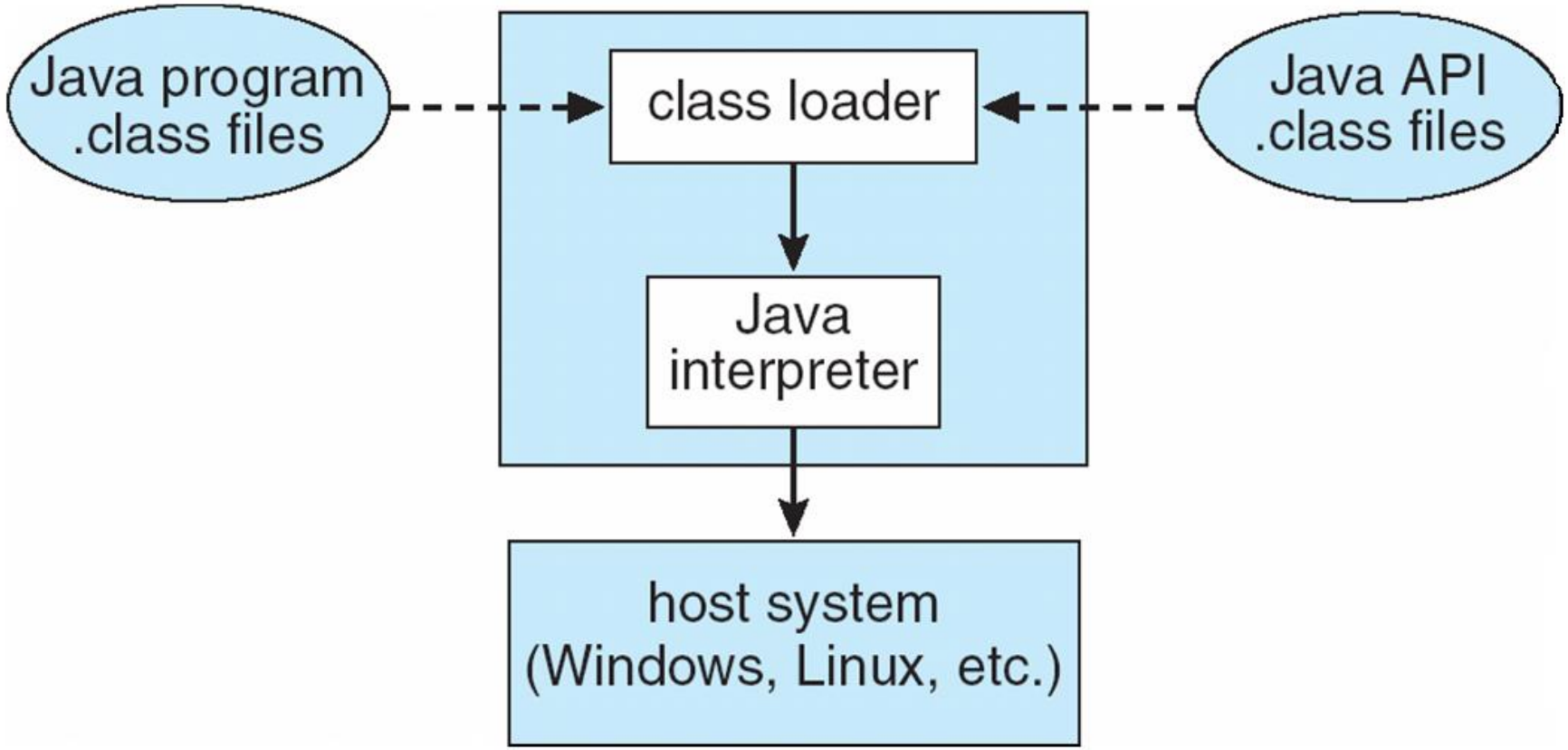
VMware Architecture



ACHARYA
BENGALURU, INDIA

Click to Edit

The Java Virtual Machine



ACHARYA
BENGALURU, INDIA

Click to Edit

THE .NETFRAMEWORK

ACHARYA
BENGALURU, INDIA

Click to Edit

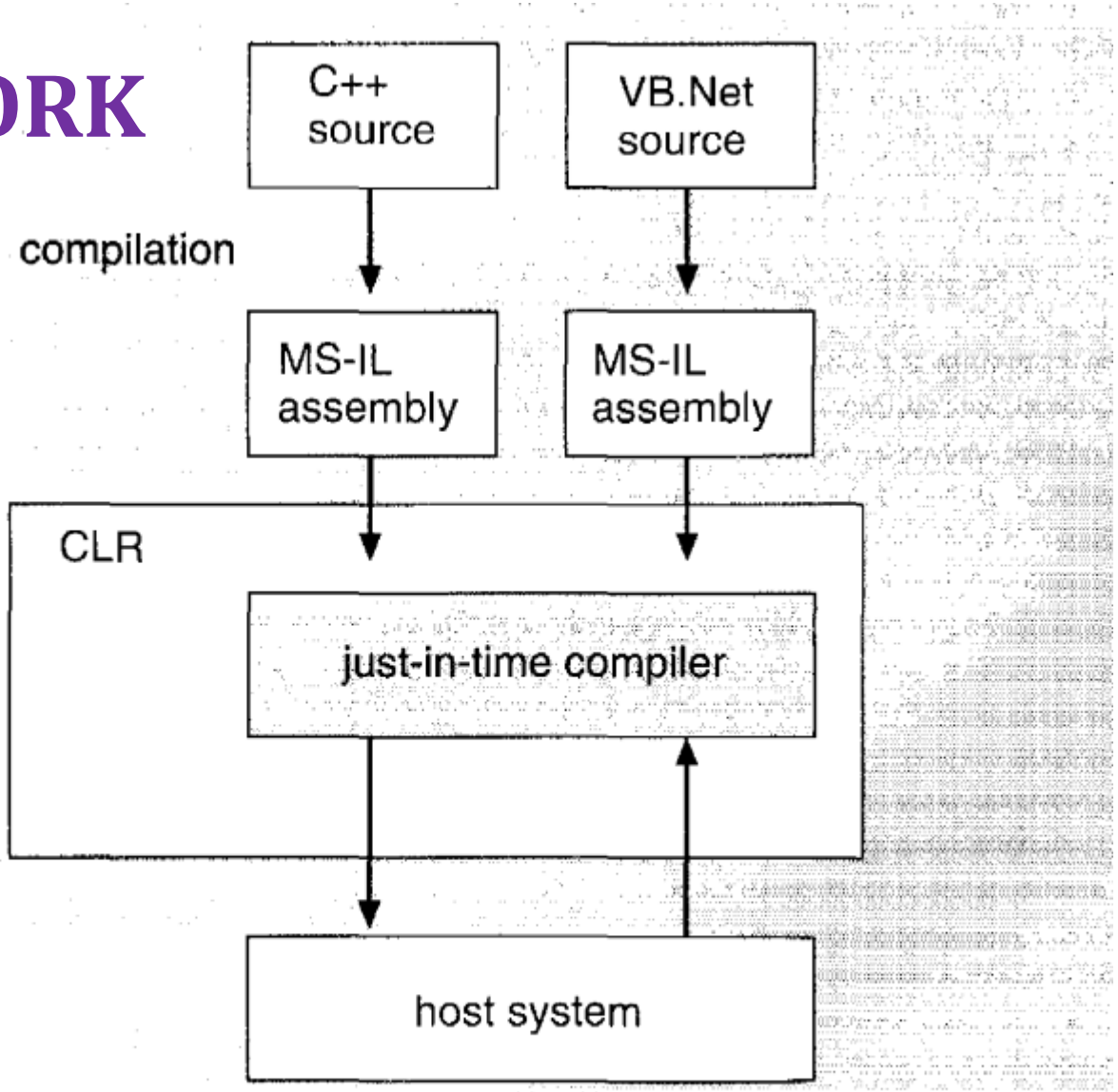


Figure 2.18 Architecture of the CLR for the .NET Framework.

Operating System Generation

- Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site

- **SYSGEN program** obtains information concerning the specific configuration of the hardware system

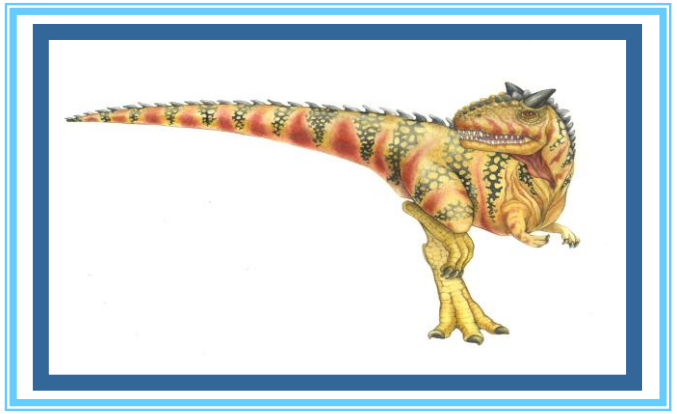
System Boot

- *Booting* – starting a computer by loading the kernel

- *Bootstrap program* – code stored in ROM that is able to locate the kernel, load it into memory, and start its execution
- **read-only memory (ROM)**
- **erasable programmable read-only memory (EPROM)**

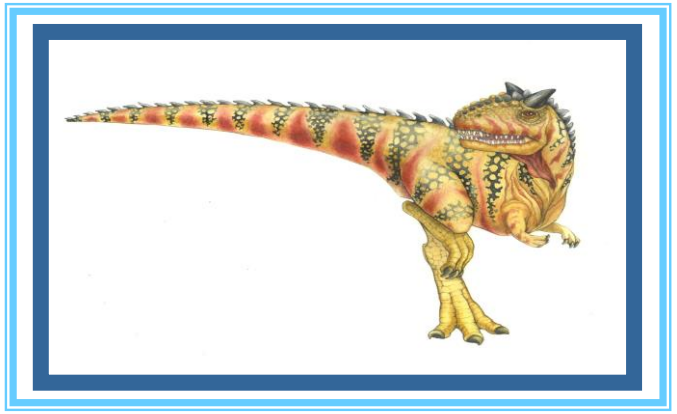
ACHARYA
BENGALURU, INDIA
Click to Edit

End of Chapter 2



Silberschatz, Galvin and Gagne ©2009

Chapter 3: Processes



Silberschatz, Galvin and Gagne ©2009

Chapter 3: Processes

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- Examples of IPC Systems
- Communication in Client-Server Systems



Click to Edit

ACHARYA
BENGALURU, INDIA

Click to Edit

Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via **a process identifier** (**pid**)
- Resource sharing
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution
  - Parent and children execute concurrently
  - Parent waits until children terminate

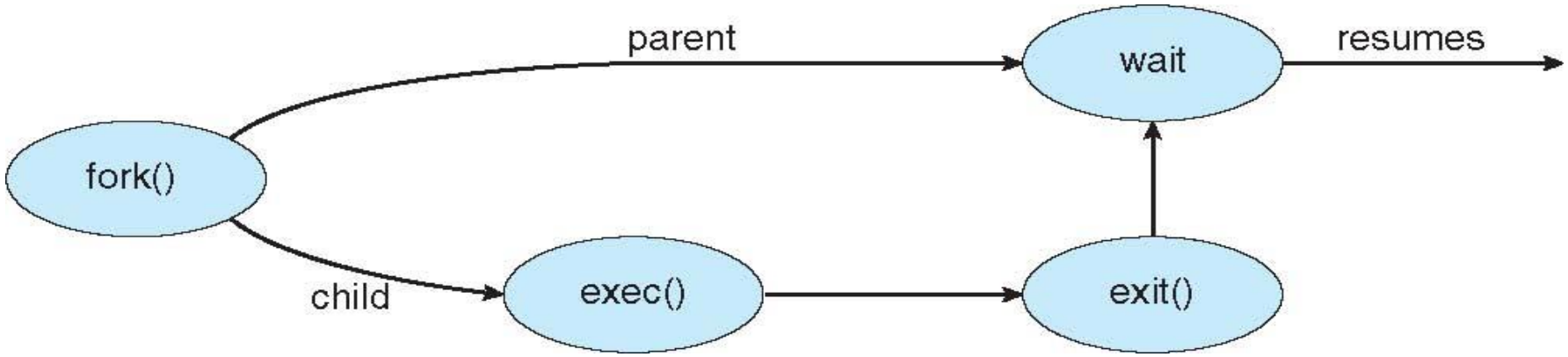
Process Creation (Cont.)



Click to Edit

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **fork** system call creates new process
  - **exec** system call used after a **fork** to replace the process' memory space with a new program

Process Creation



ACHARYA
BENGALURU, INDIA

Click to Edit

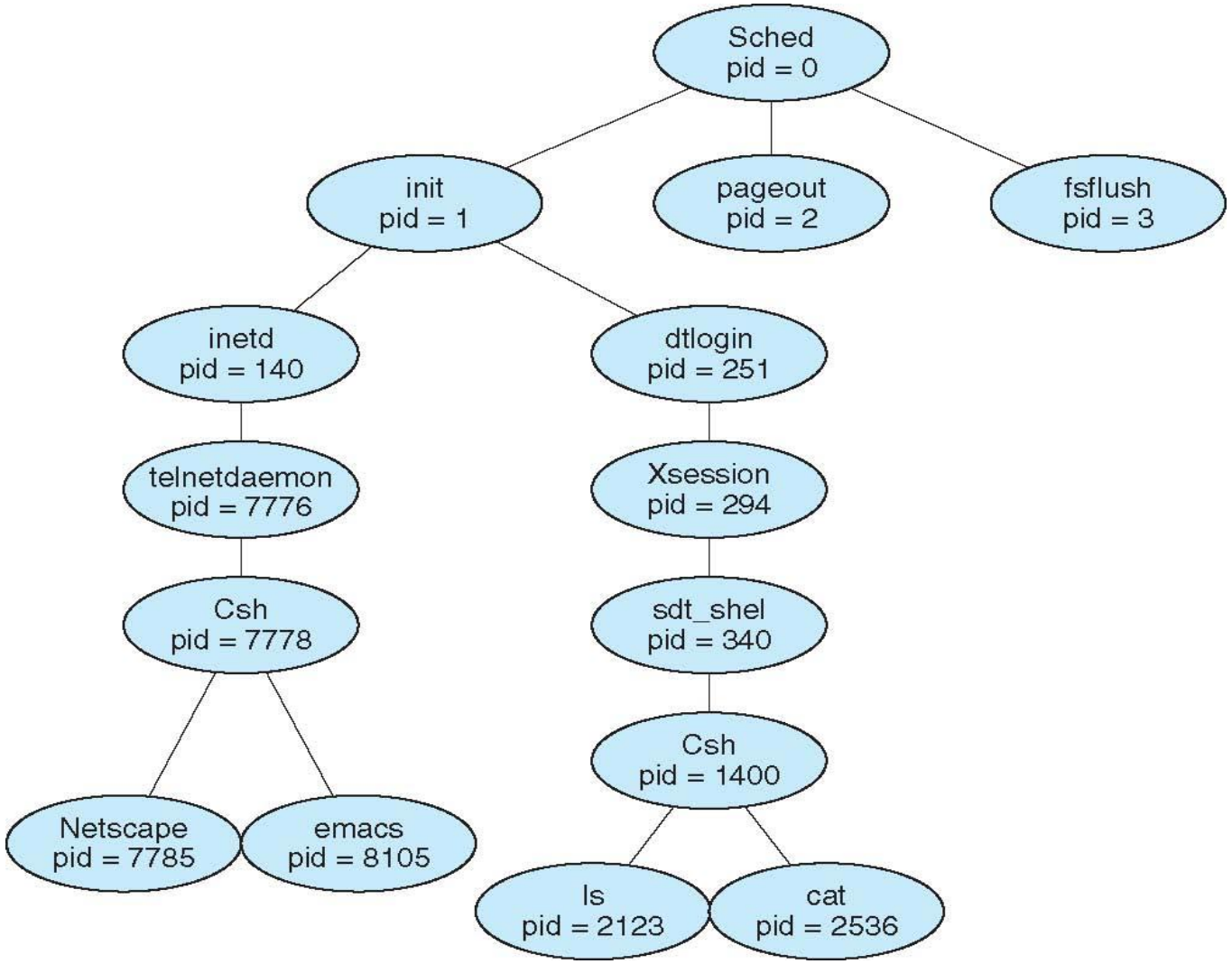
C Program Forking Separate Process



Click to Edit

```
#include <sys/types.h>
#include <studio.h>
#include <unistd.h>
int main()
{
pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
                fprintf(stderr, "Fork Failed");
                return 1;
    }
    else if (pid == 0) { /* child process */
                execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
                /* parent will wait for the child */
                wait (NULL);
                printf ("Child Complete");
    }
    return 0;
}
```

A Tree of Processes on Solaris



ACHARYA
BENGALURU, INDIA

Click to Edit

Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
  - Output data from child to parent (via **wait**)
  - Process' resources are deallocated by operating system
- Parent may terminate execution of children processes (**abort**)
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - If parent is exiting
    - Some operating systems do not allow child to continue if its parent terminates
      - All children terminated - **cascading termination**

ACHARYA
BENGALURU, INDIA

Click to Edit

Interprocess Communication

- Processes within a system may be **independent** or **cooperating**
- Cooperating processes need **interprocess communication** (**IPC**)



Click to Edit

Cooperating Processes

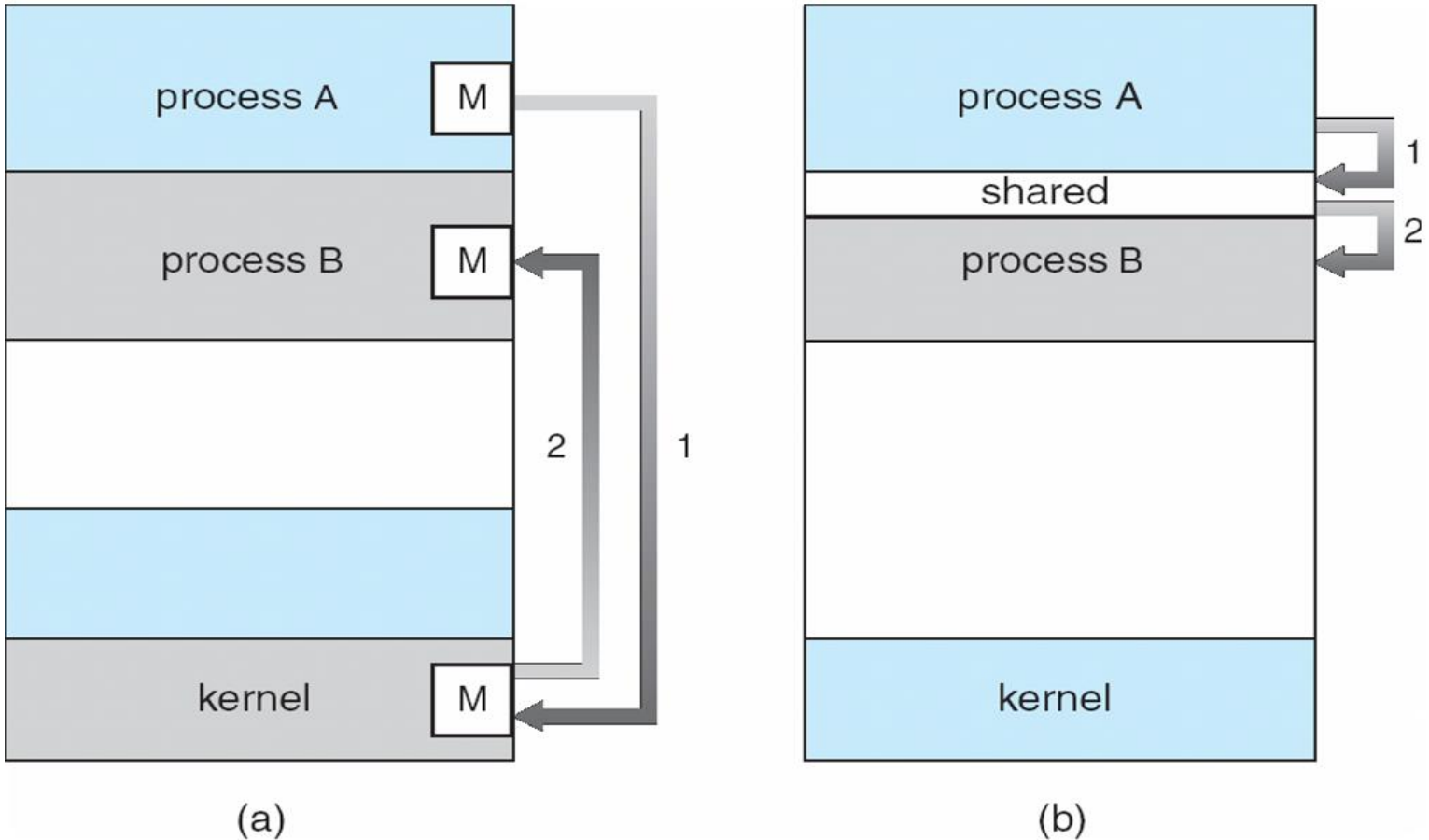


Click to Edit

- **Independent** process cannot affect or be affected by the execution of another process
- **Cooperating** process can affect or be affected by the execution of another process
- Advantages of process cooperation
  - Information sharing
  - Computation speed-up
  - Modularity
  - Convenience

Interprocess Communication

- Two models of IPC
  - Shared memory
  - Message passing



ACHARYA
BENGALURU, INDIA

Click to Edit

Producer-Consumer Problem



Click to Edit

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
- The producer and consumer must be synchronized.
- Two types of buffers can be used.
  - *unbounded-buffer* places no practical limit on the size of the buffer
  - *bounded-buffer* assumes that there is a fixed buffer size

Bounded-Buffer – Shared-Memory Solution



- Bounded buffer can be used to enable processes to share memory.
- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
   . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Bounded-Buffer – Shared-Memory Solution

- The shared buffer is implemented as a circular array with two logical pointers: in and out.
- The variable **in** points to the next free position in the buffer;
- **out** points to the first full position in the buffer.
- The buffer is empty when **in == out;**
- the buffer is full when **((in + 1) % BUFFER5IZE) == out.**

ACHARYA
BENGALURU, INDIA

Click to Edit

Bounded-Buffer – Producer



Click to Edit

```
while (true) {
  /* Produce an item */
    while ((((in + 1) % BUFFER SIZE count)  == out)
       /* do nothing -- no free buffers */
   buffer[in] = item;
   in = (in + 1) % BUFFER SIZE;
 }
```

Bounded Buffer – Consumer

```
while (true) {
     while (in == out)
          ; // do nothing -- nothing to consume

    // remove an item from the buffer
    item = buffer[out];
    out = (out + 1) % BUFFER SIZE;
return item;
 }
```



Click to Edit

Interprocess Communication – Message Passing



Click to Edit

- Mechanism for processes to communicate and to synchronize their actions
- IPC facility provides two operations:
  - **send**(*message*) – message size fixed or variable
  - **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
  - establish a *communication link* between them
  - exchange messages via send/receive
- Implementation of communication link
  - physical (e.g., shared memory, hardware bus)
  - logical (e.g., logical properties)

Implementation Questions

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?



Click to Edit

Implementation Questions

- Here are several methods for logically implementing a link and the sendO/receiveO operations:
  - Direct or indirect communication
  - Synchronous or asynchronous communication
  - Automatic or explicit buffering
- We look at issues related to each of these feahlres next.

ACHARYA
BENGALURU, INDIA

Click to Edit

Direct Communication

- **Processes must name each other explicitly:**
  - **send** (*P*, *message*) – send a message to process P
  - **receive**(*Q*, *message*) – receive a message from process Q

- **Properties of the communication link**
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link

ACHARYA
BENGALURU, INDIA
Click to Edit

Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox



Click to Edit

Indirect Communication

- Operations
  - create a new mailbox
  - send and receive messages through mailbox
  - destroy a mailbox

- Primitives are defined as:

  send(*A, message*) – send a message to mailbox A

  receive(*A, message*) – receive a message from mailbox A

ACHARYA
BENGALURU, INDIA
Click to Edit

Indirect Communication

- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Between each pair of communicating processes, there may be a number of different links, with each link corresponding to one mailbox.

ACHARYA
BENGALURU, INDIA

Click to Edit

Indirect Communication



Click to Edit

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. (Round Robin).

Synchronization

- Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** has the sender block until the message is received
  - **Blocking receive** has the receiver block until a message is available

- **Non-blocking** is considered **asynchronous**
  - **Non-blocking** send has the sender send the message and continue
  - **Non-blocking** receive has the receiver receive a valid message or null



Click to Edit

Buffering

- Queue of messages attached to the link; implemented in one of three ways
  1. Zero capacity – 0 messages
     Sender must wait for receiver
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits

ACHARYA
BENGALURU, INDIA

Click to Edit

Examples of IPC Systems - POSIX

- POSIX Shared Memory
  - Process first creates shared memory segment

```
segment id = shmget(IPC PRIVATE, size, S IRUSR
  | S IWUSR);
```

  - Process wanting access to that shared memory must attach to it

```
shared memory = (char *) shmat(id, NULL, 0);
```

  - Now the process could write to the shared memory

```
sprintf(shared memory, "Writing to shared
  memory");
```

  - When done a process can detach the shared memory from its address space

```
shmdt(shared memory);
```

ACHARYA
BENGALURU, INDIA
Click to Edit

ACHARYA
BENGALURU, INDIA

Click to Edit

```c
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the identifier for the shared memory segment */
int segment_id;=
/* a pointer to the shared memory segment */
char* shared_memory;
/* the size (in bytes) of the shared memory segment */
const int size = 4096;

   /* allocate a shared memory segment */
   segment_id = shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);

   /* attach the shared memory segment */
   shared_memory = (char *) shmat(segment_id, NULL, 0);

   /* write a message to the shared memory segment */
   sprintf(shared_memory, "Hi there!");

   /* now print out the string from shared memory */
   printf("*%s\n", shared_memory);

   /* now detach the shared memory segment */
   shmdt(shared_memory);

   /* now remove the shared memory segment */
   shmctl(segment_id, IPC_RMID, NULL);

   return 0;
}
```

Figure 3.16 C program illustrating POSIX shared-memory API.

Examples of IPC Systems - Mach



Click to Edit

- Mach communication is message based
  - Most communication in Mach-including most of the system calls and all intertask information is carried out by messages.
  - Messages are sent to and received from mailboxes, called ports in Mach.
  - When a task is created, two special mailboxes-the Kernel mailbox and the Notify mailbox-are also created.
  - Only three system calls needed for message transfer msg_send(), msg_receive(), msg_rpc()
  - Mailboxes needed for commuication, created via port_allocate()

Examples of IPC Systems - Mach

- Mach communication is message based
  - Mailboxes needed for commuication, created via port_allocate()
  - The port_allocate () system call creates a new mailbox and allocates space for its queue of messages.
  - The maximum size of the message queue defaults to eight messages.
  - The messages themselves consist of a fixed-length header followed by a variable-length data portion.
  - The header indicates the length of the message and includes two mailbox names.

ACHARYA
BENGALURU, INDIA

Click to Edit

Examples of IPC Systems - Mach



Click to Edit

- Mach communication is message based
  - The send and receive operations themselves are flexible. For instance, when a message is sent to a mailbox, the mailbox may be full.
  - If the mailbox is not full, the message is copied to the mailbox, and the sending thread continues.
  - If the mailbox is full, the sending thread has four options:
    1. Wait indefinitely until there is room in the mailbox.
    2. Wait at most n milliseconds.
    3. Do not wait at all but rather return immediately.
    4. Temporarily cache a message.

Examples of IPC Systems – Windows XP



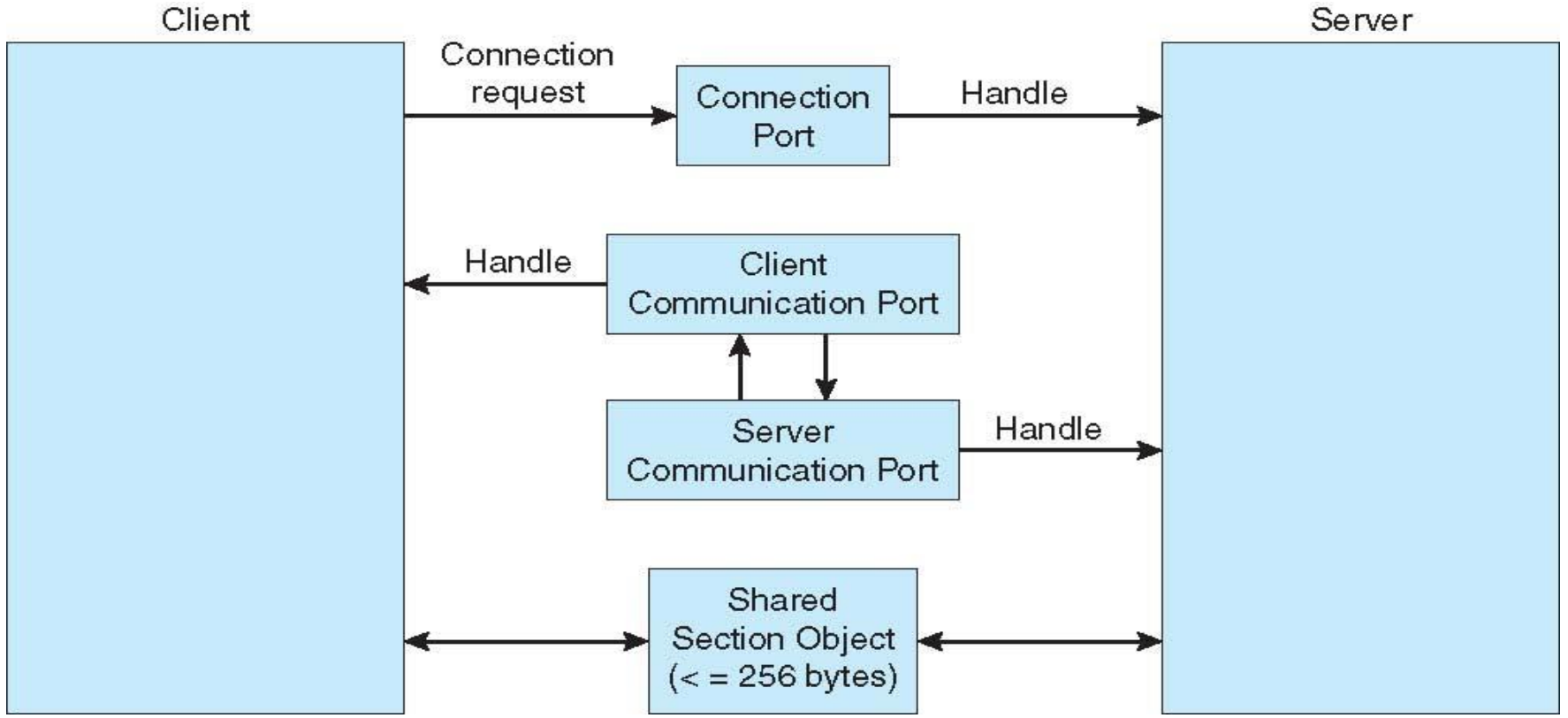
Click to Edit

- Message-passing centric via **local procedure call (LPC)** facility
  - Only works between processes on the same system
  - Uses ports (like mailboxes) to establish and maintain communication channels
  - Communication works as follows:
    - The client opens a handle to the subsystem's connection port object.
    - The client sends a connection request.
    - The server creates two private communication ports and returns the handle to one of them to the client.
    - The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.

Local Procedure Calls in Windows XP

ACHARYA
BENGALURU, INDIA

Click to Edit



Communications in Client-Server Systems



Click to Edit

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

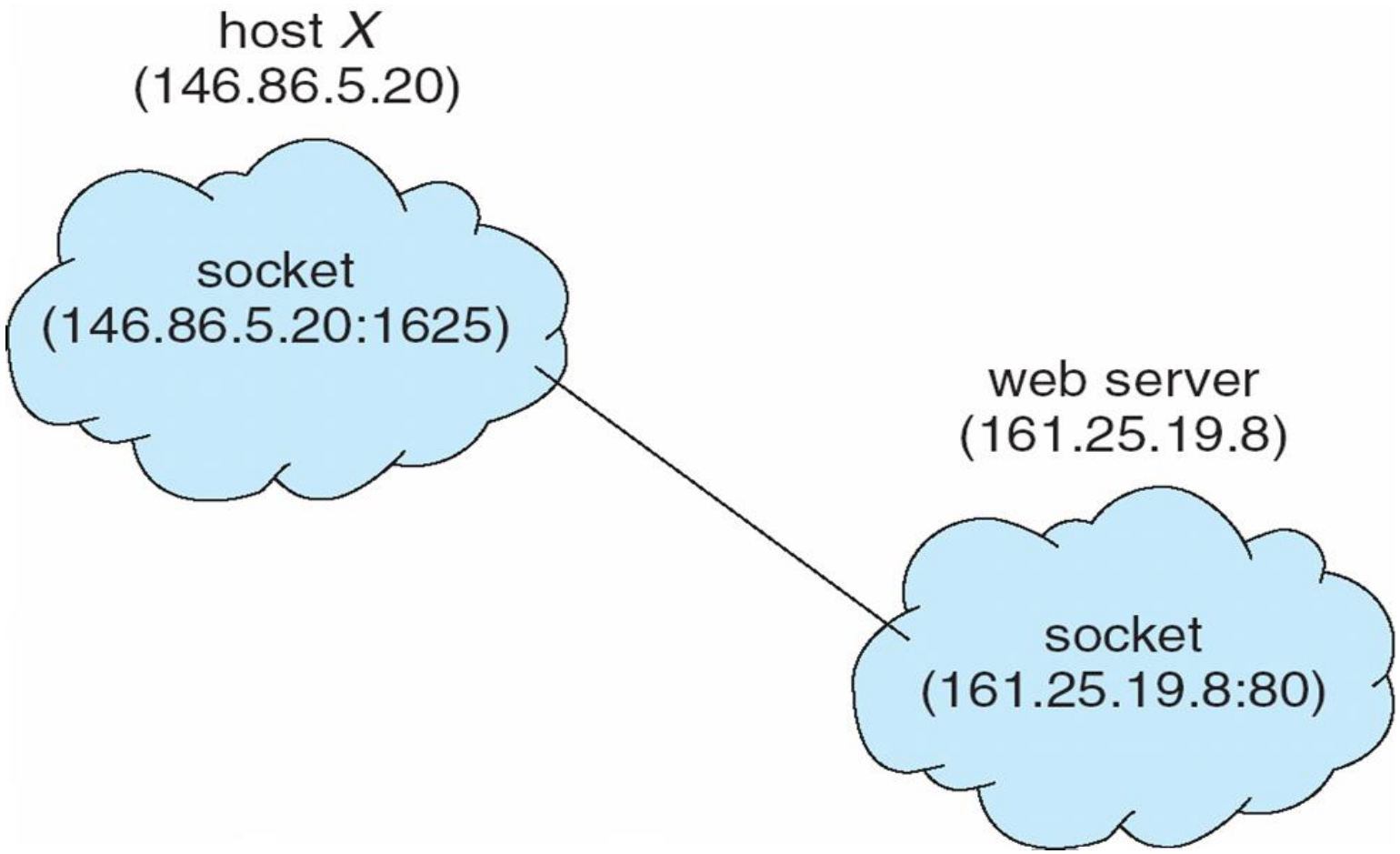
Sockets

- A **socket** is defined as an *endpoint for communication*
- Concatenation of IP address and port
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets

ACHARYA
BENGALURU, INDIA

Click to Edit

Socket Communication



ACHARYA
BENGALURU, INDIA

Click to Edit

Sockets

- Java provides three different types of sockets.
- Connection-oriented (TCP) sockets are implemented with the Socket class.
- Connectionless (UDP) sockets use the DatagramSocket class.
- Finally, the MulticastSocket class is a subclass of the DatagramSocket class.
- A multicast socket allows data to be sent to multiple recipients.



Click to Edit

Sockets

ACHARYA
BENGALURU, INDIA

Click to Edit

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         // now listen for connections
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            // write the Date to the socket
            pout.println(new java.util.Date().toString());

            // close the socket and resume
            // listening for connections
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```

Figure 3.19 Date server.

Sockets

ACHARYA
BENGALURU, INDIA

Click to Edit

```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args) {
      try {
         //make connection to server socket
         Socket sock = new Socket("127.0.0.1",6013);

         InputStream in = sock.getInputStream();
         BufferedReader bin = new
            BufferedReader(new InputStreamReader(in));

         // read the date from the socket
         String line;
         while ( (line = bin.readLine()) != null)
            System.out.println(line);

         // close the socket connection
         sock.close();
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```

Figure 3.20 Date client.

Remote Procedure Calls

- Remote Procedure Call (RPC) is a communication protocol that allows a computer program to invoke a procedure or function in another address space, typically on a remote computer or server, as if it were a local procedure call.

ACHARYA
BENGALURU, INDIA

Click to Edit

Remote Procedure Calls

- The RPC process involves two key components:
- the client and the server.
- The client program initiates the RPC by making a request to execute a procedure on the server.
- The server program receives the request, executes the procedure, and sends back the results to the client.
- This interaction between the client and server is handled by the RPC framework.

ACHARYA
BENGALURU, INDIA

Click to Edit

Remote Procedure Calls

ACHARYA
BENGALURU, INDIA

Click to Edit

- Here's a simplified overview of how the RPC process works:

- **Client-side stub:** The client program calls a local stub procedure, which acts as a proxy for the remote procedure. The stub accepts the parameters from the client program and packages them into a message.

- **Parameter marshalling:** The parameters of the remote procedure call need to be serialized into a format suitable for transmission over the network. The stub converts the parameters into a standard format **(external data representation)** that both the client and server understand.

- **Message passing:** The client-side runtime system sends the message containing the serialized parameters to the server over the network.

Remote Procedure Calls

- **Server-side stub:** The server-side runtime system receives the message and forwards it to the appropriate server stub, which unpacks the parameters from the message.

- **Procedure execution:** The server stub locates the actual procedure or function to be executed based on the request and invokes it, passing the unpacked parameters.

- **Result marshalling:** If the remote procedure has a return value, the server stub serializes the result into a standard format.

ACHARYA
BENGALURU, INDIA

Click to Edit

Remote Procedure Calls

ACHARYA
BENGALURU, INDIA

Click to Edit

- **Message passing:** The server-side runtime system sends the message containing the serialized result back to the client over the network.
- **Client-side stub:** The client-side runtime system receives the message and forwards it to the client stub.
- **Result unmarshalling:** The client stub unpacks the result from the message and converts it into a format usable by the client program.
- **Client program:** Finally, the client program receives the result of the remote procedure call and continues its execution.

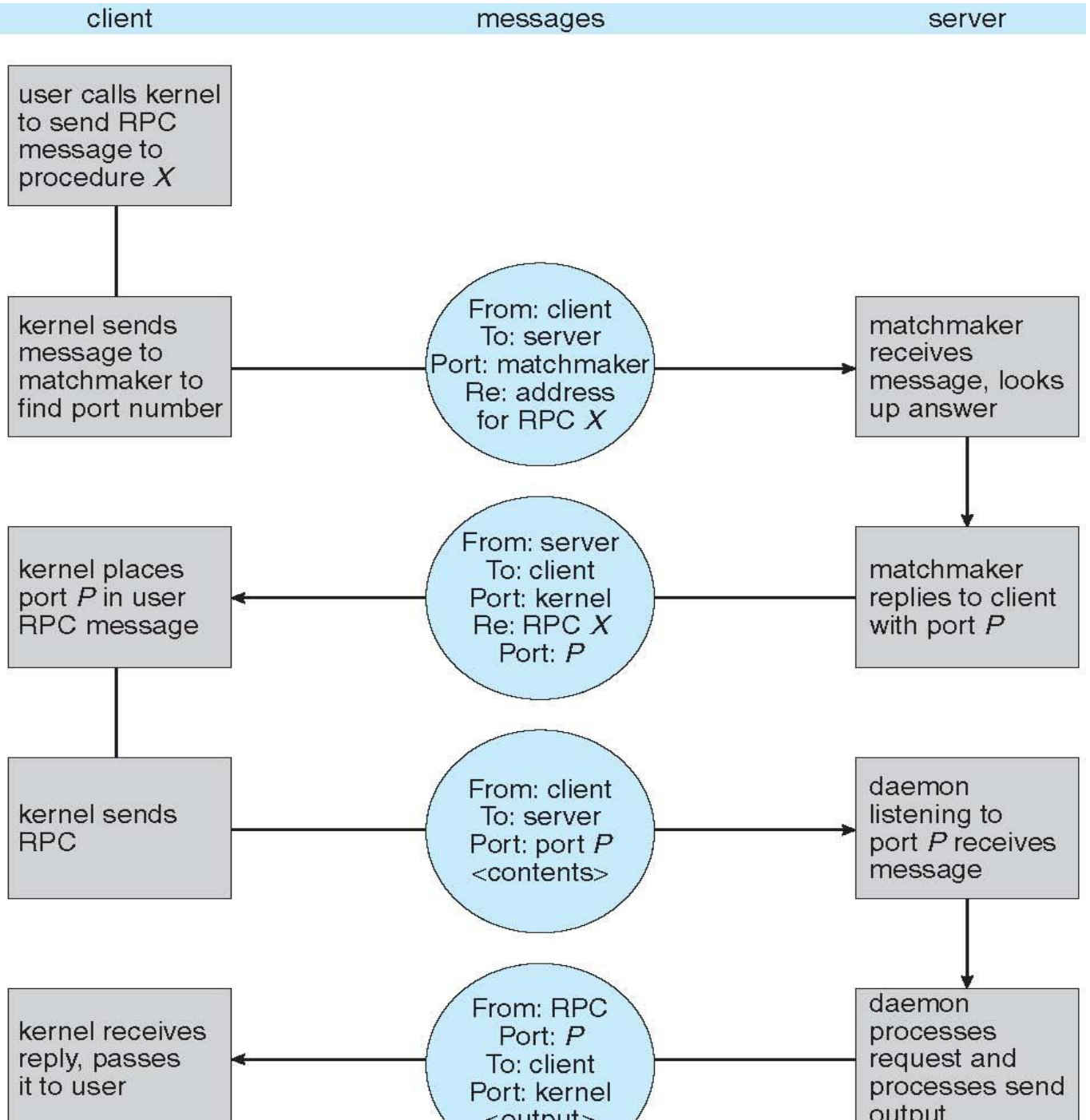
Remote Procedure Calls

- Another important issue involves the semantics of a call. Whereas local procedure calls fail only under extreme circumstances, RPCs can fait or be duplicated and executed more than once, as a result of common network errors.
- One way to address this problem is for the operating system to ensure that messages are acted on exactly once, rather than at most once.



Click to Edit

Execution



ACHARYA
BENGALURU, INDIA

Click to Edit

Remote Method Invocation

ACHARYA
BENGALURU, INDIA

Click to Edit

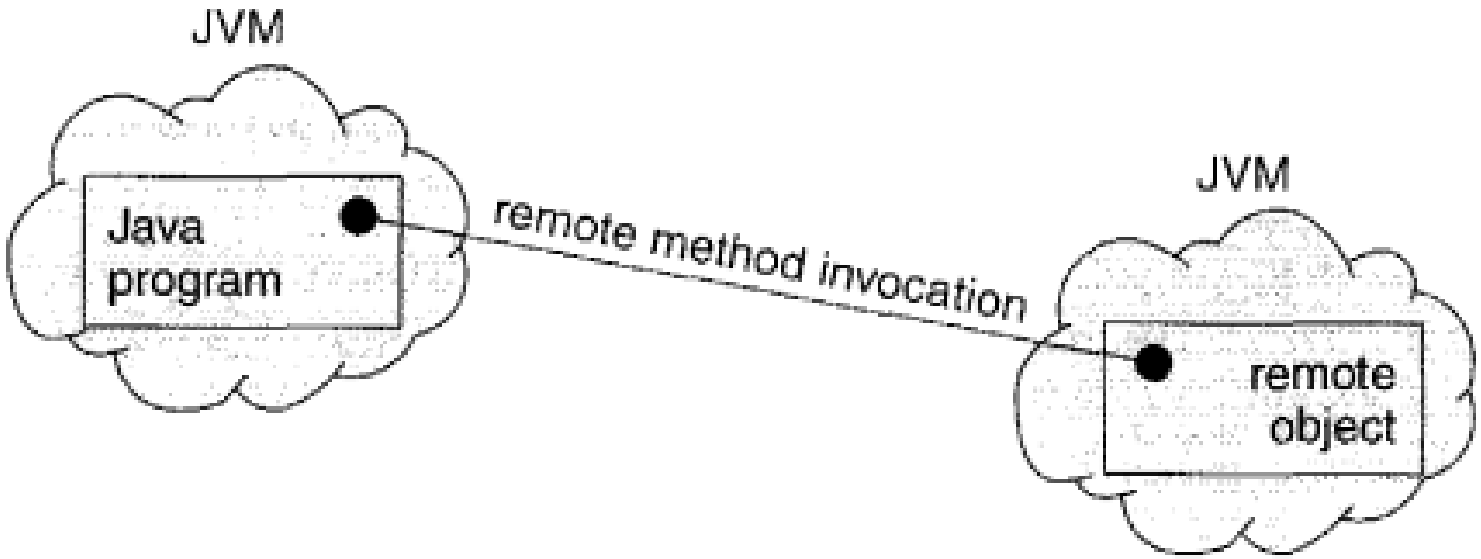


Figure 3.22 Remote method invocation.

Remote Method Invocation

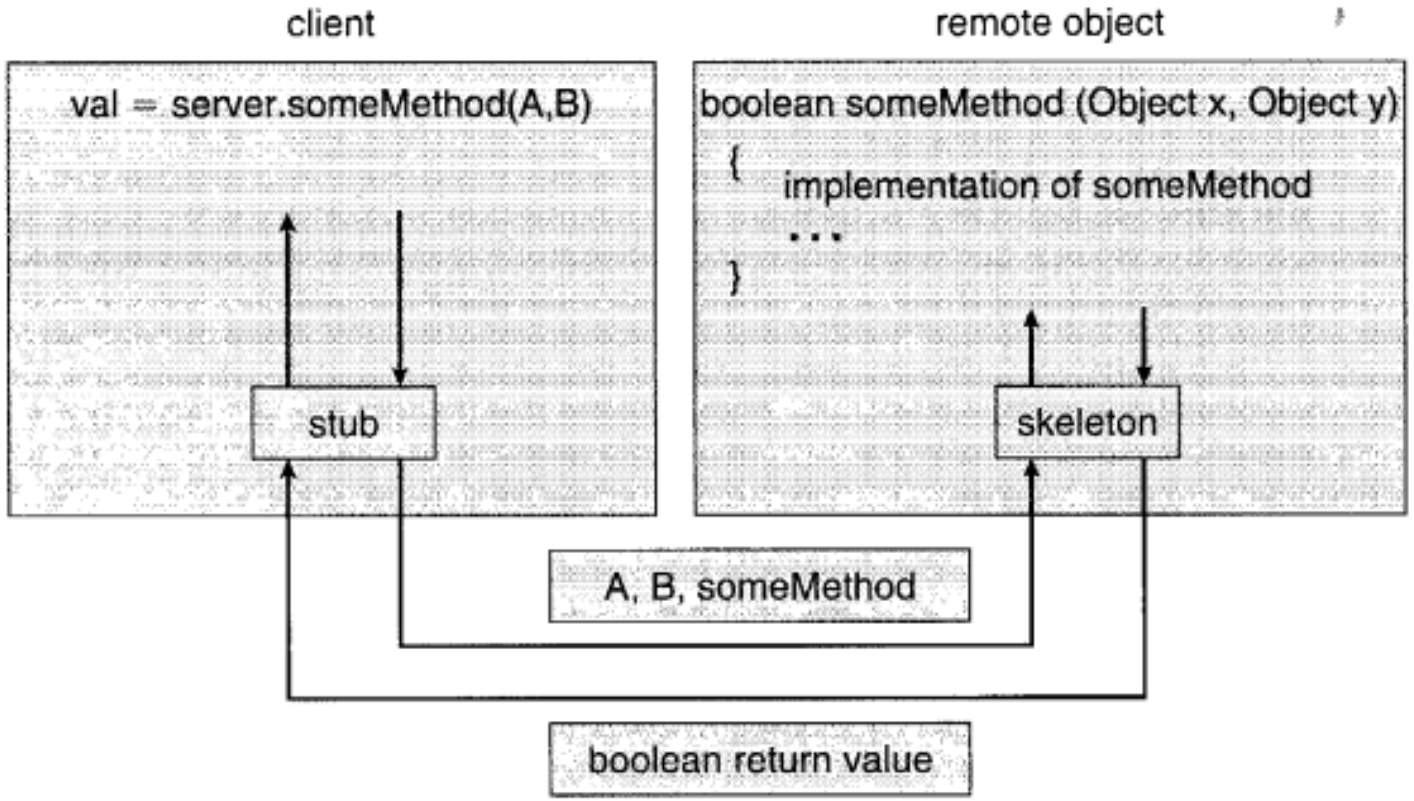
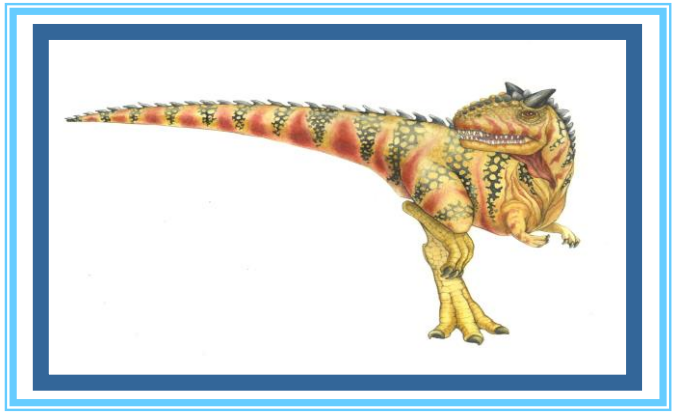


Figure 3.23 Marshalling parameters.

ACHARYA
BENGALURU, INDIA

Click to Edit

End of Chapter 3



Silberschatz, Galvin and Gagne ©2009